1. Portada

Proyecto Integrador de Sistemas Operacionales

Reto 2 — Infraestructura Web Segura con LEMP Stack

Nginx · PHP-FPM 8.5 · MariaDB sobre Ubuntu Server 26.04 LTS

Integrantes: Juan Diego López · Luis Camilo Vivas Cruz · Dylan Mancipe

Curso: Sistemas Operacionales (2026-2)

Docente: Ing. Samuel Agustín Galindo Pinzón

Universidad: Universidad El Bosque — Facultad de Ingeniería de Sistemas

Fecha de entrega: 9 de noviembre de 2026

Tabla de contenido

2. Resumen ejecutivo.....3
3. Introducción.....3
4. Marco conceptual.....4
5. Análisis y comparación de alternativas.....5
6. Diseño de la arquitectura.....6
7. Implementación.....7
8. Gestión de usuarios y permisos.....10
9. Hardening implementado.....12
10. Monitoreo y logging.....14
11. Incidente simulado como estudio de caso.....15
12. Pruebas de validación desde el segundo nodo.....18
13. Conclusiones.....19
14. Recomendaciones y trabajo futuro.....20
15. Referencias.....20
16. Anexos.....21
Anexo A. Código PHP de la aplicación.....21
Anexo B. Archivos de configuración finales.....48
Anexo C. Scripts SQL y datos reales.....53
Anexo D. Bitácora técnica.....54
Anexo E. Script de monitoreo (/usr/local/bin/monitor_lemp.sh).....58

2. Resumen ejecutivo

Montamos una infraestructura web segura sobre Ubuntu Server 26.04.1 LTS con el stack LEMP: Nginx 1.28.3 como servidor web, PHP 8.5 ejecutado por PHP-FPM y MariaDB 11.8.6. En un mismo servidor (*srv-lemp*) funcionan dos virtual hosts con contenido distinto y conectados a la misma base de datos: *sitio1.local*, un Panel de Administradores que permite crear, editar y eliminar usuarios, y *sitio2.local*, un Portal de Lectores de solo consulta. Los dos se sirven únicamente por HTTPS (TLS 1.2/1.3, certificado autofirmado con SAN) y todo acceso por HTTP recibe un 301.

Cada sitio tiene su propio pool de PHP-FPM con un usuario de sistema distinto (*www-sitio1*, *www-sitio2*) y *open_basedir*, de modo que el código de un sitio no puede leer los archivos del otro. En MariaDB, *web_admin* tiene SELECT/INSERT/UPDATE/DELETE y *web_read* solo SELECT; comprobamos que *web_read* recibe el error 1142 al intentar escribir. El servidor solo acepta tráfico que llega por la VPN Tailscale (UFW: 22, 80 y 443 en *tailscale0*), SSH funciona solo con llaves y sin root, hay política de contraseñas, actualizaciones automáticas y encabezados de seguridad en Nginx sin revelar versiones.

Como incidente simulamos un 502 Bad Gateway en un solo sitio provocado por permisos mal asignados al socket del pool de *sitio2*. Lo diagnosticamos descartando primero que PHP-FPM estuviera caído y confirmando en el *error.log* el errno 13 (Permission denied); la corrección fue devolver el socket al usuario de Nginx. Todas las pruebas se validaron desde un segundo nodo (el portátil de un integrante) conectado por Tailscale.

3. Introducción

3.1 Contexto

Una aplicación web dinámica no depende solo de su código: necesita un servidor web, un intérprete, una base de datos, una red y una política de seguridad que funcionen juntos. El reto pide desplegar una aplicación así con distintos niveles de acceso administrativo y dejarla protegida, lo que nos obligó a aplicar casi todos los temas del curso: procesos y servicios (systemd, workers de Nginx y PHP-FPM), usuarios y permisos, sockets, red y firewall, registros y diagnóstico.

3.2 Objetivo

Desplegar dos virtual hosts sobre Nginx con PHP-FPM y MariaDB, con una aplicación conectada a la base de datos, usuarios de base de datos con privilegios diferenciados, HTTPS con redirección forzada, firewall restrictivo, hardening general y del servicio, monitoreo, un incidente diagnosticado y validación desde un segundo nodo.

3.3 Alcance

- Servidor *srv-lemp* (VM VirtualBox) con Ubuntu Server 26.04.1 LTS; segundo nodo: portátil del equipo.
- Nginx, PHP 8.5-FPM y MariaDB; dos vhosts (*sitio1.local*, *sitio2.local*) con la aplicación PHP del equipo.
- HTTPS con certificado autofirmado (SAN) y redirección 301; un pool PHP-FPM aislado por sitio.
- Usuarios de BD *web_admin* / *web_read* y perfiles de aplicación *admin* / *lector*.
- UFW restringido a Tailscale, SSH por llaves, política de contraseñas, actualizaciones automáticas.
- Hardening de Nginx, PHP y MariaDB; logs por sitio y script de monitoreo programado; incidente simulado.

3.4 Fuera de alcance

Dominio público y certificado de una CA reconocida, alta disponibilidad y balanceo, replicación y copias de seguridad de MariaDB, monitoreo con alertas (Prometheus/Grafana), WAF y despliegue en nube de producción. Tampoco modificamos el código de la aplicación: el equipo decidió desplegarla tal cual (ver 8.3 y 14).

4. Marco conceptual

LEMP. Linux + Nginx (“engine-x”) + MariaDB/MySQL + PHP. Cada pieza hace una sola cosa: Nginx recibe las conexiones, PHP-FPM ejecuta el código y MariaDB guarda los datos; se comunican por sockets.

Virtual host. Un bloque *server* de Nginx. Con un solo proceso y una sola IP se atienden varios sitios: Nginx mira el nombre pedido (cabecera Host en HTTP o SNI en el saludo TLS) y lo compara con *server_name* para elegir el bloque.

FastCGI y PHP-FPM. Nginx no interpreta PHP; le pasa la petición por el protocolo FastCGI a PHP-FPM. PHP-FPM tiene un proceso maestro (root) que administra pools; cada pool lanza workers con el usuario que se le configure.

Socket Unix y permisos. Un archivo especial (*/run/php/…sock*) por el que dos procesos del mismo equipo se comunican. Para conectarse hay que tener permiso de escritura sobre él, igual que con cualquier archivo: dueño, grupo y modo cuentan.

TLS/HTTPS y certificado autofirmado. TLS cifra la conexión y el certificado prueba la identidad del servidor. Uno autofirmado cifra igual, pero ninguna autoridad lo respalda, por eso el navegador advierte y *curl* necesita *-k*. La extensión SAN permite que un mismo certificado sirva para *sitio1.local* y *sitio2.local*.

Privilegios en MariaDB. Un usuario es la pareja usuario@host y solo puede hacer lo que le den los GRANT. Si intenta algo no concedido, MariaDB responde con el error 1142 (comando denegado).

UFW. Interfaz simple sobre el firewall del kernel (nftables). Permite reglas por puerto y por interfaz, como “aceptar 443 solo si entra por tailscale0”.

Tailscale/WireGuard. VPN de malla basada en WireGuard. Cada equipo obtiene una IP 100.x en la interfaz *tailscale0*; el tráfico entre nodos va cifrado aunque estén en redes distintas.

5. Análisis y comparación de alternativas

5.1 Nginx frente a Apache

Comparamos los dos servidores antes de instalar, con criterios que afectan directamente a este caso (dos sitios PHP pequeños en una VM de 2 GB, detrás de una VPN):

Criterio	Nginx	Apache HTTP Server
Modelo de procesamiento	Pocos workers con un bucle de eventos asíncrono; cada worker atiende miles de conexiones.	MPM: prefork (un proceso por conexión), worker (hilos) o event (hilos + eventos para keep-alive).
Memoria con muchas conexiones	Casi constante: las conexiones inactivas no ocupan un proceso/hilo.	Crece con las conexiones en prefork/worker; event lo mejora.
Ejecución de PHP	Siempre externa por FastCGI (PHP-FPM): procesos y usuarios separados del servidor web.	mod_php dentro del proceso de Apache (todo con el mismo usuario) o PHP-FPM vía proxy_fcgi.
Configuración	Centralizada en */etc/nginx*; un *nginx -t* valida todo antes de recargar.	Admite *.htaccess* por directorio: flexible, pero se lee en cada petición y reparte la configuración.
Rol de proxy inverso	Diseñado como proxy/terminador TLS (proxy_pass, fastcgi_pass, upstream).	Lo hace con mod_proxy, pero no es su uso típico.

5.2 Decisión

Elegimos Nginx. Para este caso pesaron tres cosas: (1) obliga a usar PHP-FPM, lo que nos permitió dar a cada sitio su propio usuario de sistema y su propio socket — con mod_php los dos sitios correrían con el mismo usuario; (2) toda la configuración vive en un solo árbol y se valida con *nginx -t*, sin *.htaccess* dispersos que un atacante con escritura en el webroot pudiera aprovechar; (3) consume poca memoria, importante en una VM de 2 GB que comparte el portátil con el resto del trabajo. Apache sería preferible si necesitáramos *.htaccess* (hosting compartido donde cada cliente configura su carpeta), que no es el caso.

5.3 Un pool PHP-FPM compartido frente a uno por sitio

Ubuntu trae un único pool *www* con el usuario *www-data*, el mismo de Nginx. Con él, si un sitio tiene una vulnerabilidad, el código malicioso puede leer el otro sitio. Con un pool por sitio cada uno tiene su usuario, su socket y su *open_basedir*; el costo es algo más de memoria (dos grupos

de workers) y más archivos que mantener. Elegimos un pool por sitio y desactivamos el pool *www*.

6. Diseño de la arquitectura

La solución tiene dos nodos unidos por Tailscale. El servidor solo deja entrar tráfico que llega por la VPN; dentro, Nginx reparte por nombre a cada vhost, cada vhost tiene su pool PHP-FPM y ambos usan MariaDB local.

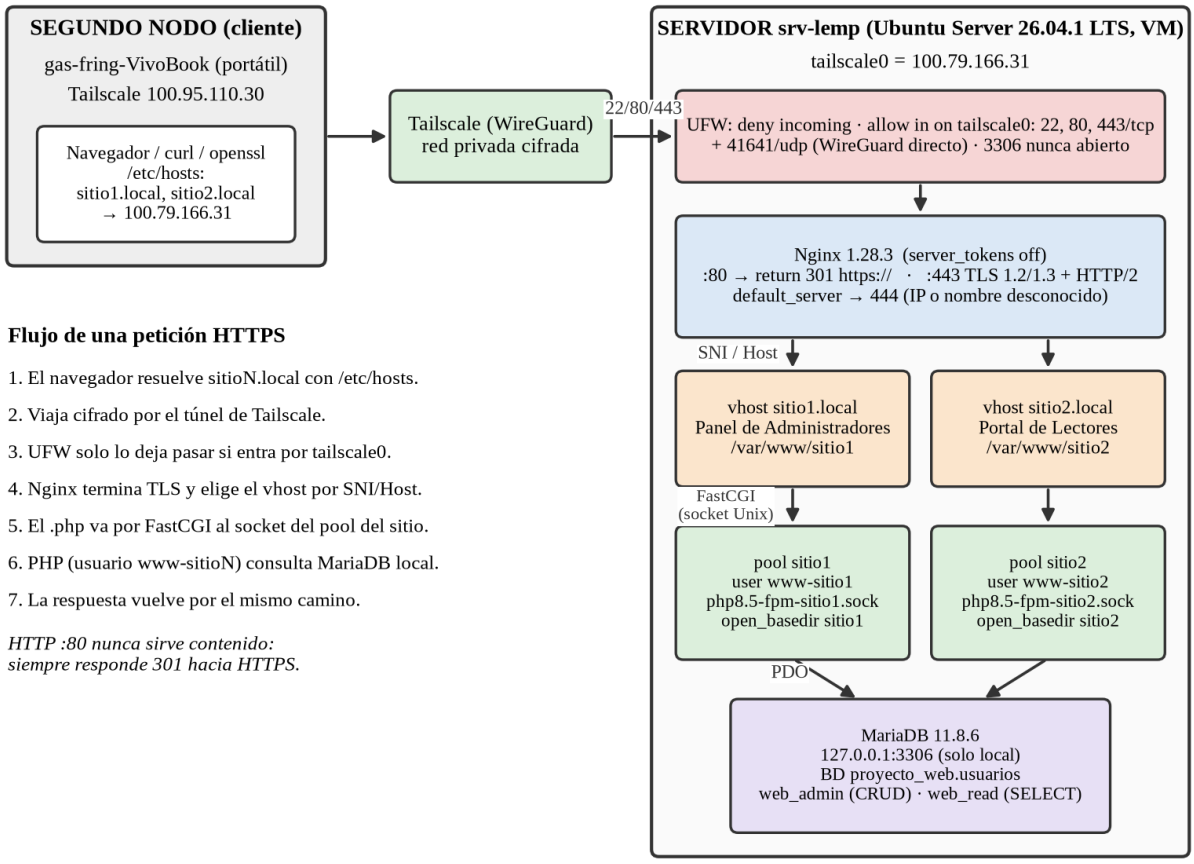


Figura 1. Arquitectura (Paso Cero): nodos, IP de Tailscale, firewall, vhosts, pools, sockets y base de datos, con el flujo de una petición HTTPS.

6.1 Puertos y protocolos

Puerto	Protocolo	Servicio	Exposición
22	TCP	SSH (administración, solo llaves)	Solo por *tailscale0*
80	TCP	HTTP: responde siempre 301 a HTTPS	Solo por *tailscale0*
443	TCP	HTTPS (TLS 1.2/1.3, HTTP/2)	Solo por *tailscale0*
41641	UDP	WireGuard de Tailscale (conexión directa entre nodos)	Abierto: solo transporta el túnel cifrado
3306	TCP	MariaDB	Solo 127.0.0.1; nunca abierto en UFW

7. Implementación

7.1 Preparación del servidor

Creamos una VM dedicada (*srv-lemp*, 2 GB de RAM, 2 vCPU, 25 GB) con la ISO oficial de Ubuntu Server 26.04.1, verificada con su suma SHA-256. Descartamos reutilizar una VM anterior porque alojaba otros proyectos (un clúster, NFS, otras bases de datos): un firewall que solo permita 22/80/443 los habría roto, y sus puertos habrían contaminado la evidencia de "exponer únicamente lo necesario". Después fijamos la zona horaria a *America/Bogota* (para que las evidencias muestren la hora real) y actualizamos el sistema.

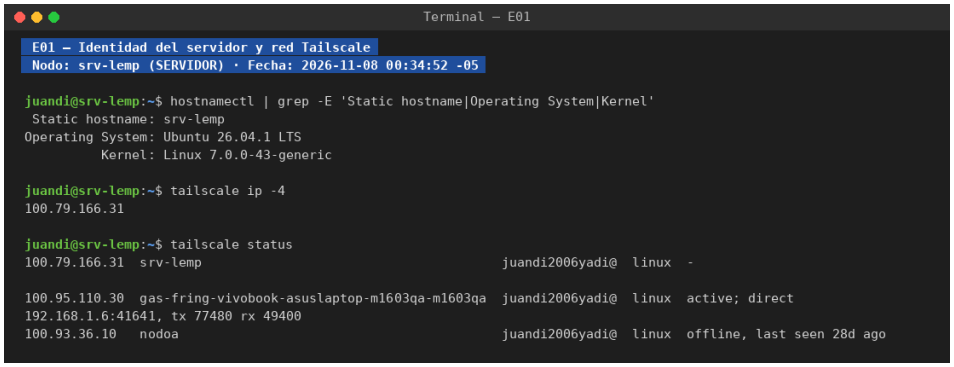


Figura 2. Identidad del servidor y estado de la red Tailscale (E01; salida real de capturar.sh, con hostname y fecha)

7.2 Conectividad entre nodos

Instalamos Tailscale desde su repositorio oficial y unimos el servidor a la red privada del equipo; recibió la IP 100.79.166.31. El segundo nodo es el portátil de un integrante (100.95.110.30). En la consola de Tailscale desactivamos la expiración de la llave del servidor: si vence, el nodo sale de la red hasta que alguien inicie sesión otra vez, algo inaceptable en un servidor (nos ocurrió con una VM anterior).

7.3 Nginx, PHP-FPM y MariaDB

Instalamos los tres servicios desde los repositorios de Ubuntu (*nginx*, *php8.5-fpm*, *php8.5-mysql*, *php8.5-cli*, *mariadb-server*) y los dejamos habilitados al arranque: quedaron Nginx 1.28.3, PHP 8.5.4 y MariaDB 11.8.6. *php8.5-mysql* aporta el controlador PDO que usa la aplicación.

7.4 Base de datos

Creamos la base *proyecto_web* (utf8mb4) con la tabla *usuarios(id, nombre, correo, password, rol ENUM('admin','lector'))*. Las contraseñas de la aplicación se guardan como hash bcrypt con *password_hash()*, nunca en texto plano. El script completo y los datos reales están en el Anexo C.

7.5 Certificado y virtual hosts

Generamos un certificado autofirmado RSA 2048 con SAN para los dos nombres, con la llave privada en modo 600. Cada vhost tiene dos bloques *server*: el del puerto 80 solo contiene *return 301 https://$host$request_uri;* (nunca entrega contenido sin cifrar) y el del 443 sirve el sitio y envía los *.php* a su pool. Los habilitamos con enlaces simbólicos en *sites-enabled*, retiramos el sitio *default* y validamos con *nginx -t* antes de recargar. Las configuraciones finales están en el Anexo B.

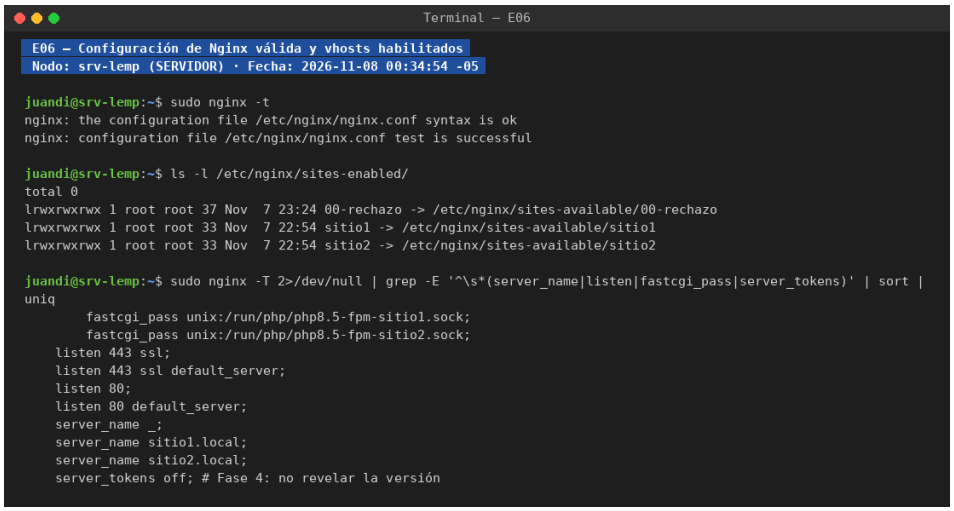


Figura 3. Configuración de Nginx válida y vhosts habilitados (E06; salida real de capturar.sh, con hostname y fecha)

Un error que tuvimos: los dos sitios respondían 403 Forbidden. La causa era un *umask 077* que pusimos para proteger el archivo de credenciales de la BD y que quedó activo para el resto del script, así que */var/www/sitio1* y */var/www/sitio2* se crearon con modo 700 y el usuario de Nginx no podía entrar. Lo corregimos con *chmod 755* y limitando el *umask* a un subshell.

7.6 Resolución de nombres en el segundo nodo

Como *sitio1.local* y *sitio2.local* no existen en ningún DNS, en el segundo nodo agregamos a */etc/hosts* la línea *100.79.166.31 sitio1.local sitio2.local*. Los dos nombres apuntan a la misma IP; es Nginx quien los separa por SNI/Host. Por eso un *ping sitio2.local* muestra la respuesta como *sitio1.local*: *ping* imprime el primer nombre asociado a esa IP en */etc/hosts*, no es un error.

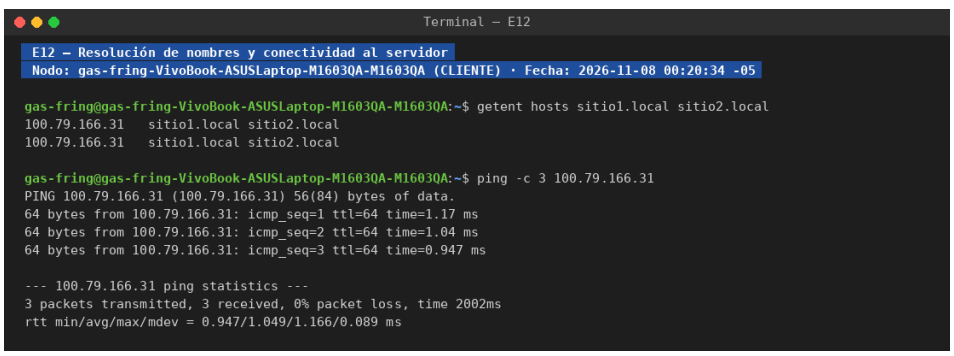


Figura 4. Resolución de nombres y conectividad desde el segundo nodo (E12; salida real de capturar.sh, con hostname y fecha)

7.7 Aplicación web conectada a la base de datos

La aplicación es la que desarrolló el equipo, desplegada sin modificaciones (código completo en el Anexo A). *sitio1.local* es el Panel de Administradores: inicio de sesión de usuarios con rol *admin* y altas, ediciones y bajas de usuarios. *sitio2.local* es el Portal de Lectores: inicio de sesión de usuarios con rol *lector* y consulta de la lista. Ambas usan PDO con sentencias preparadas, *password_verify()* y *htmlspecialchars()*. Los archivos quedaron con dueño root y modo 644: los procesos PHP pueden leerlos pero no modificarlos.

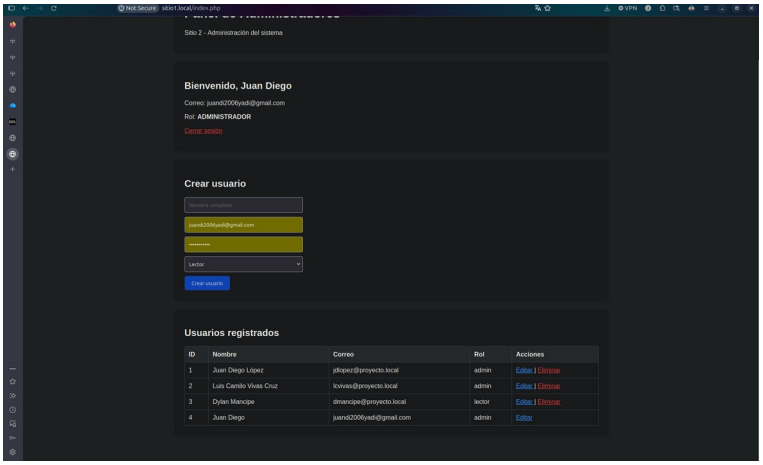


Figura 5. Panel de Administradores (sitio1.local) mostrando usuarios leídos de MariaDB (E14; captura del navegador)

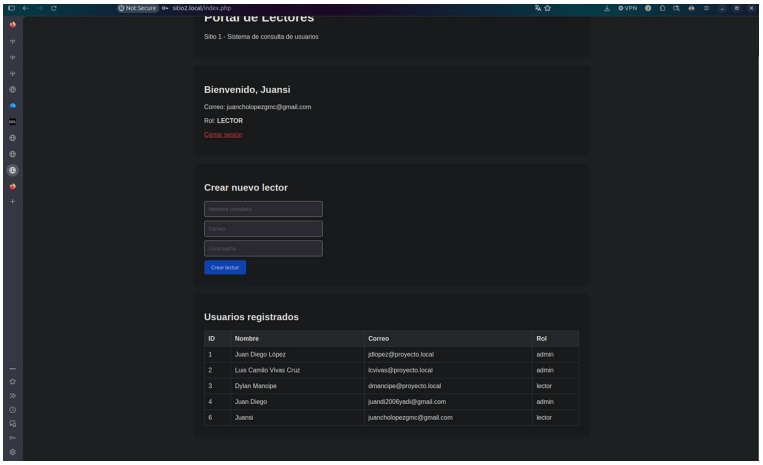


Figura 6. Portal de Lectores (sitio2.local) con sesión de un usuario lector (E15; captura del navegador)

7.8 Un pool PHP-FPM por sitio

Creamos los usuarios de sistema *www-sitio1* y *www-sitio2* (sin shell ni home) y un pool para cada uno. En cada pool, *user*/*group* definen con qué usuario corre el PHP del sitio; *listen* define su socket propio y *listen.owner*/*listen.group* = *www-data* permiten que Nginx se conecte a él;

open_basedir limita los archivos que el PHP puede abrir a su propio webroot, el directorio de sesiones y */tmp*. Desactivamos el pool por defecto.

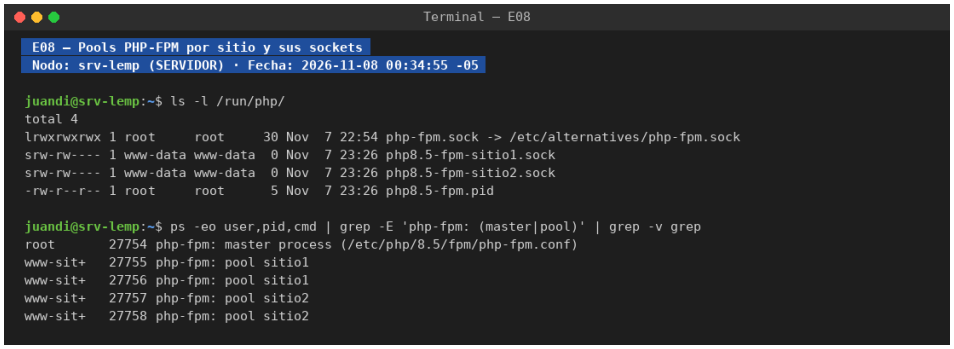


Figura 7. Pools PHP-FPM por sitio, sus usuarios y sus sockets (E08; salida real de capturar.sh, con hostname y fecha)

8. Gestión de usuarios y permisos

8.1 Perfiles definidos

Capa	Perfil	Nivel de acceso	Dónde se aplica
Base de datos	*web_admin@localhost*	SELECT, INSERT, UPDATE, DELETE sobre *proyecto_web.**; sin privilegios globales	GRANT en MariaDB
Base de datos	*web_read@localhost*	Solo SELECT sobre *proyecto_web.**	GRANT en MariaDB
Aplicación	rol *admin*	Entra al Panel de Administradores; crea, edita y elimina usuarios	Sesión PHP + columna *rol*
Aplicación	rol *lector*	Entra al Portal de Lectores; solo consulta	Sesión PHP + columna *rol*
Sistema	*www-sitio1* / *www-sitio2*	Ejecutan el PHP de su sitio; cada uno solo ve su webroot	Pools PHP-FPM + open_basedir
Sistema	*juandi*	Único usuario con SSH (llave), con sudo	*AllowUsers* en sshd

8.2 Evidencia de que los privilegios se respetan

Con *web_read*, la consulta funciona pero toda escritura es rechazada por el motor con el error 1142, incluso un DROP: el permiso se aplica en MariaDB, no depende de la aplicación.

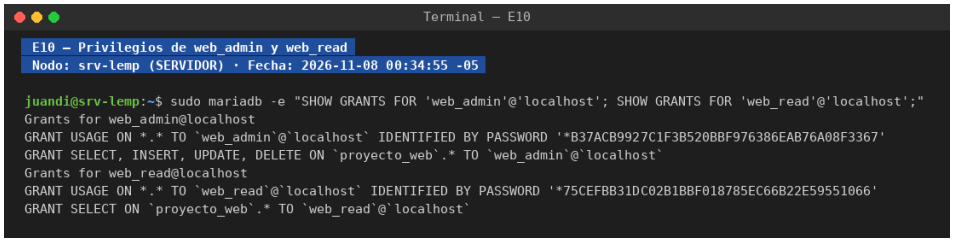


Figura 8. Privilegios de web_admin y web_read (SHOW GRANTS) (E10; salida real de capturar.sh, con hostname y fecha)

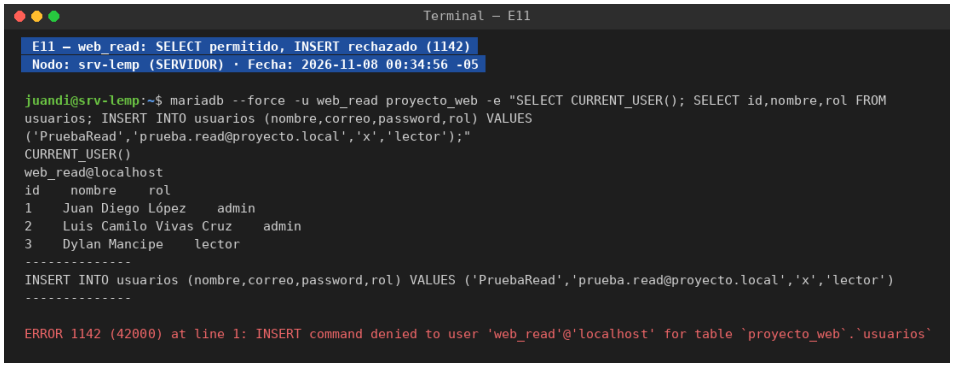


Figura 9. web_read: SELECT permitido; INSERT, DELETE y DROP rechazados con ERROR 1142 (E11; salida real de capturar.sh, con hostname y fecha)

El aislamiento entre sitios también se probó: un script temporal en *sitio2* (ejecutado como *www-sitio2*) intentó leer el código de *sitio1* y *open_basedir* lo impidió. El script se borró después de la prueba.

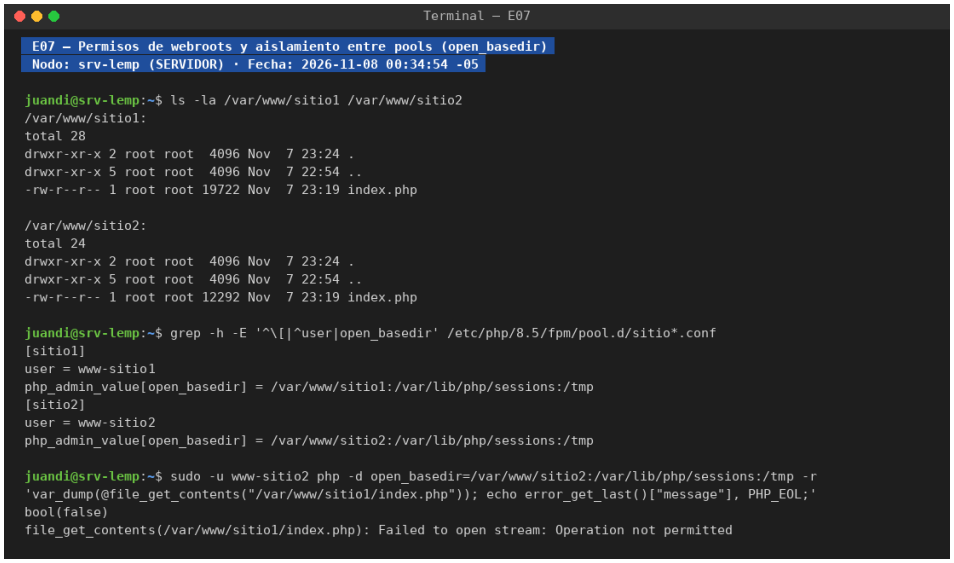


Figura 10. Aislamiento entre pools: el PHP de sitio2 no puede leer archivos de sitio1 (E07; salida real de capturar.sh, con hostname y fecha)

8.3 Limitación conocida

Por decisión del equipo la aplicación se desplegó tal cual: ambos sitios se conectan a MariaDB con *web_admin* y la contraseña está escrita en el código, y el Panel incluye un formulario público para crear administradores. La separación de privilegios de la base de datos queda demostrada por consola (8.2), pero no la usa todavía el Portal de Lectores. Las correcciones propuestas están en la sección 14.

9. Hardening implementado

9.1 Hardening general del servidor

Medida	Implementación	Evidencia
Firewall	UFW deny incoming; 22/80/443 solo *in on tailscale0*; 41641/udp para WireGuard	E03
Root sin SSH	*PermitRootLogin no*	E04, E05
Autenticación segura	Solo llaves ed25519: *PasswordAuthentication no, KbdInteractiveAuthentication no, MaxAuthTries 3, AllowUsers juandi*	E04, E05
Política de contraseñas	*libpam-pwquality*: *minlen = 12, minclass = 3*	E04
Puerto por defecto	Se mantuvo el 22 (ver justificación)	E03
Sistema actualizado	*apt full-upgrade* (0 pendientes) + *unattended-upgrades* diario	E02

Sobre el puerto de SSH: la guía pide cambiar puertos por defecto “cuando sea razonable”. Aquí no lo es: el 22 solo es alcanzable desde la VPN (desde fuera ni siquiera aparece), así que moverlo no reduce la superficie de ataque, y en Ubuntu 26.04 SSH se activa por *ssh.socket*, donde cambiar el puerto exige tocar también la unidad de systemd. Para no quedarnos sin acceso aplicamos todo en orden: primero confirmamos el login por llave por la IP de Tailscale, luego agregamos las reglas, activamos UFW y probamos una sesión nueva antes de cerrar la anterior.

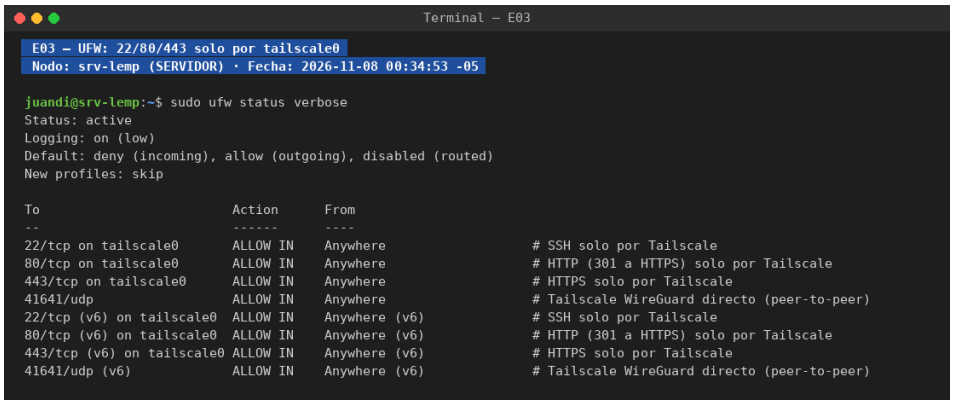


Figura 11. UFW: solo 22/80/443 por tailscale0 (E03; salida real de capturar.sh, con hostname y fecha)

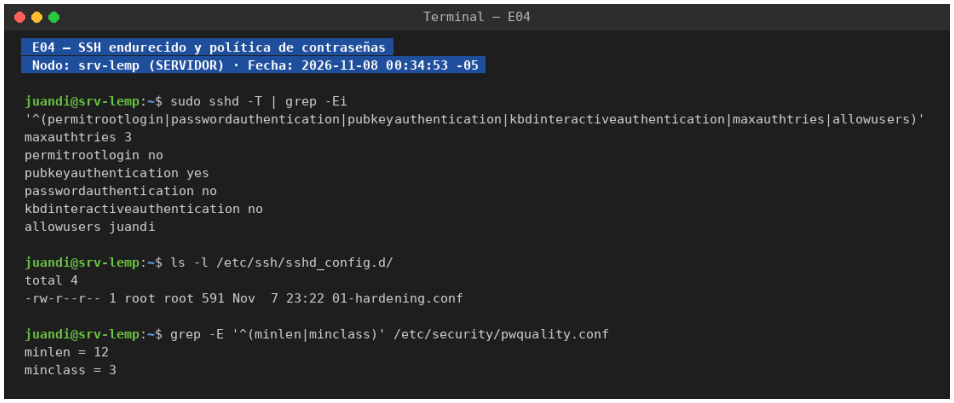


Figura 12. SSH endurecido y política de contraseñas (E04; salida real de capturar.sh, con hostname y fecha)

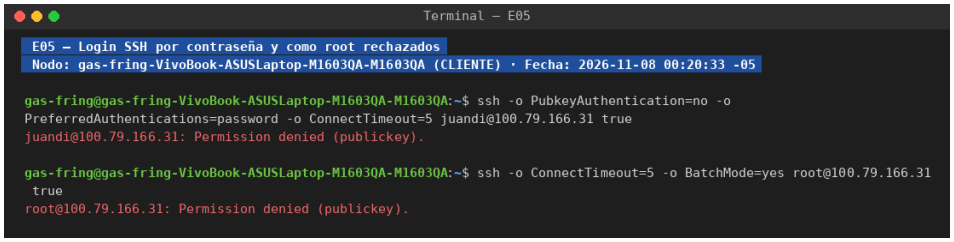


Figura 13. Desde el segundo nodo: login por contraseña y como root rechazados (E05; salida real de capturar.sh, con hostname y fecha)

9.2 Hardening de Nginx

- **Versión oculta:** *server_tokens off* (Ubuntu 26.04 trae *server_tokens build;* en *nginx.conf*; repetir la directiva en *conf.d* da “duplicate”, así que la cambiamos en su sitio).
- **TLS:** solo TLS 1.2 y 1.3; HTTP/2 activo.
- **Encabezados:** HSTS, *X-Frame-Options DENY*, *X-Content-Type-Options nosniff*, *Referrer-Policy* y CSP. La CSP permite *'unsafe-inline'* en estilos y scripts porque la aplicación usa *<style>* y un *onclick="confirm()"* en línea; sin eso, “Eliminar” borraría sin pedir confirmación.
- **Archivos ocultos:** *location ~ /\.* → 403 (protege *.env*, *.git*).
- **Abuso:** límite de 10 peticiones/s por IP (ráfaga 20) y cuerpo máximo de 2 MB.
- **Servidor por defecto:** cualquier petición por IP o con un nombre desconocido recibe 444 (se cierra sin respuesta) y en 443 se rechaza el saludo TLS; así no se revela qué sitios existen.
- **Contenido:** no existe *phpinfo.php* (404); los webroots son de root con 755/644.

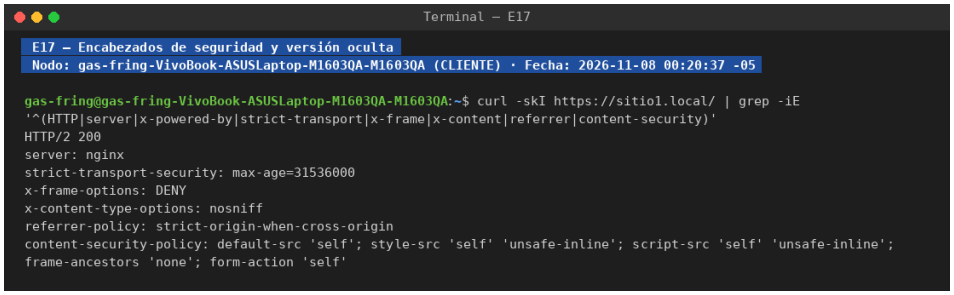


Figura 14. Encabezados de seguridad presentes y Server sin versión (E17; salida real de capturar.sh, con hostname y fecha)



Figura 15. Protocolos TLS: 1.0 y 1.1 rechazados por el servidor, 1.2 y 1.3 aceptados (E18; salida real de capturar.sh, con hostname y fecha)

9.3 Hardening de PHP y MariaDB

- PHP (*99-hardening.ini*): *expose_php = Off*, *display_errors = Off* (los errores van al log), cookies de sesión *secure*, *HttpOnly*, *SameSite=Strict* y *use_strict_mode*.
- MariaDB: *bind-address = 127.0.0.1* (el 3306 no existe hacia la red), sin usuarios anónimos ni root remoto, usuarios de aplicación con privilegios solo sobre *proyecto_web*.

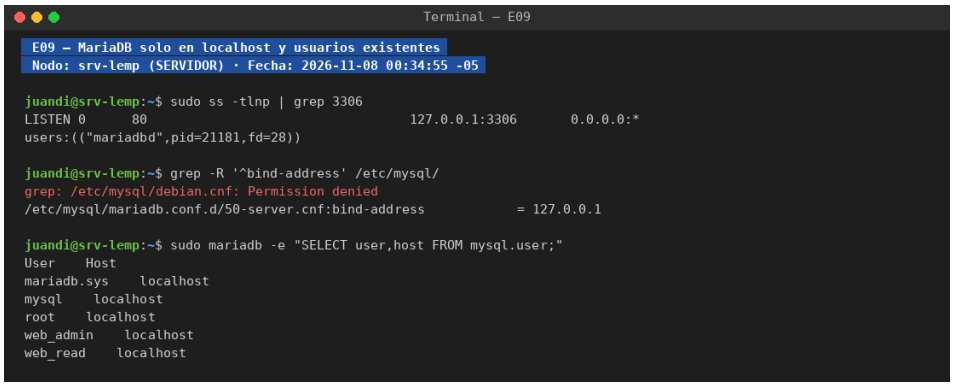


Figura 16. MariaDB solo en localhost y usuarios existentes (E09; salida real de capturar.sh, con hostname y fecha)

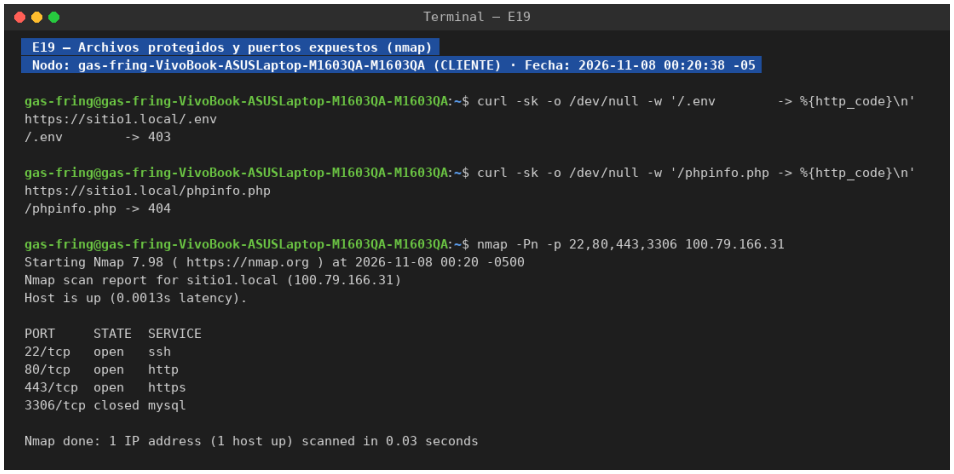


Figura 17. Archivos protegidos (403/404) y puertos expuestos vistos desde el segundo nodo (E19; salida real de capturar.sh, con hostname y fecha)

10. Monitoreo y logging

Cada vhost escribe su propio *access.log* y *error.log*, con un formato propio (*lemp*) que añade el tiempo total de la petición (*rt*) y lo que tardó PHP-FPM en responder (*urt*); así se distingue si la lentitud está en Nginx o en la aplicación. Además usamos *journalctl -u php8.5-fpm* para los eventos del servicio.

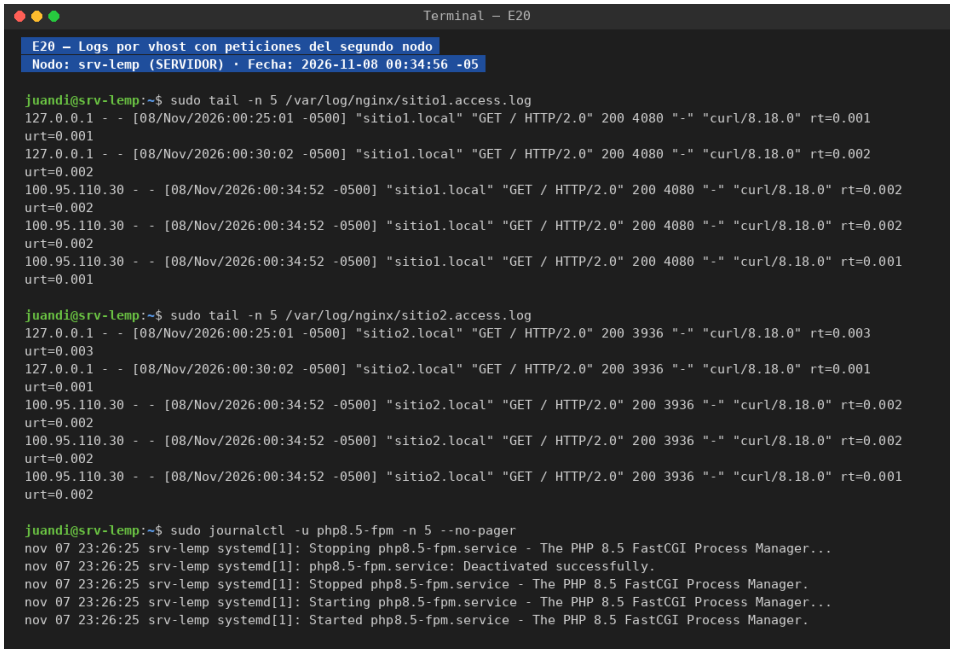


Figura 18. Logs por vhost con peticiones del segundo nodo (100.95.110.30) y tiempos de respuesta (E20; salida real de capturar.sh, con hostname y fecha)

Escribimos *monitor_lemp.sh* (Anexo E): revisa que los tres servicios estén activos, que existan los dos sockets, que cada sitio responda 200 y cuenta los errores recientes; termina con *RESUMEN=OK* o *RESUMEN=ALERTA* (código de salida 1). Corre cada 5 minutos desde */etc/cron.d/lemp-monitor* y su log rota cada semana con logrotate. Durante el incidente el monitor marcó la alerta (sección 11).

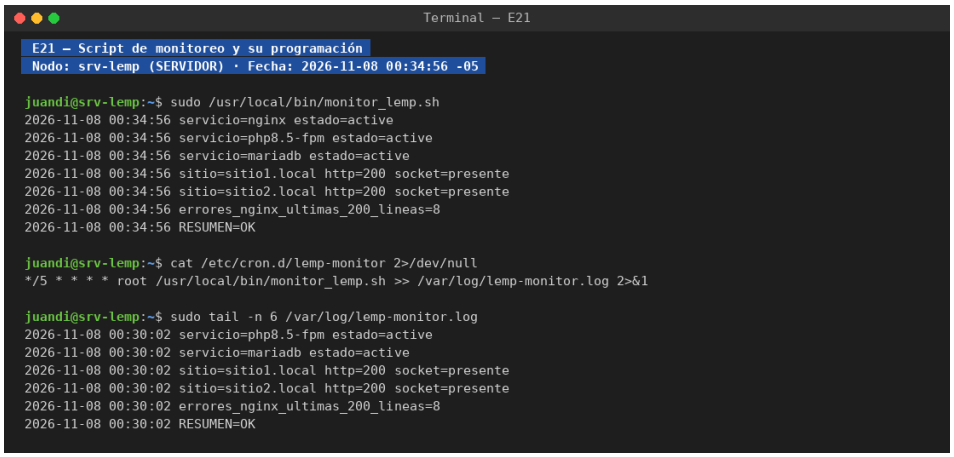


Figura 19. Script de monitoreo y su programación en cron (E21; salida real de capturar.sh, con hostname y fecha)

11. Incidente simulado como estudio de caso

En una primera versión del proyecto el incidente fue detener PHP-FPM; lo reemplazamos porque la hipótesis era obvia (el servicio estaba parado) y porque tumbaba los dos sitios a la vez. El

nuevo incidente es más realista: un administrador, queriendo “aislar mejor” *sitio2*, cambió el dueño de su socket al usuario del pool.

11.1 Provocación

```
$ sudo cp -a /etc/php/8.5/fpm/pool.d/sitio2.conf /root/respaldos/sitio2.conf.bak.incidente
$ sudo sed -i "s/^listen.owner = www-data/listen.owner = www-sitio2/; s/^listen.group = www-data/listen.group = www-sitio2/" /etc/php/8.5/fpm/pool.d/sitio2.conf
$ sudo php-fpm8.5 -t
[07-Nov-2026 23:26:01] NOTICE: configuration file /etc/php/8.5/fpm/php-fpm.conf test is successful
$ sudo systemctl restart php8.5-fpm
```

La sintaxis es válida (*php-fpm8.5 -t* no avisa nada): el error es de lógica, no de sintaxis.

11.2 Síntoma observado

Desde el segundo nodo, *sitio1* sigue respondiendo 200 y *sitio2* devuelve 502 Bad Gateway. Que falle un solo sitio ya descarta la red, el firewall, el certificado y Nginx caído, porque los dos los comparten.

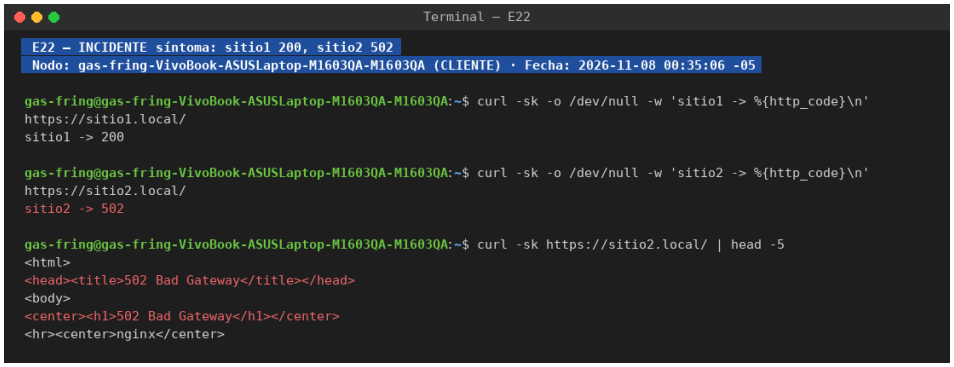


Figura 20. Síntoma desde el cliente: sitio1 200, sitio2 502 (E22; salida real de capturar.sh, con hostname y fecha)

11.3 Hipótesis 1: PHP-FPM está caído — descartada

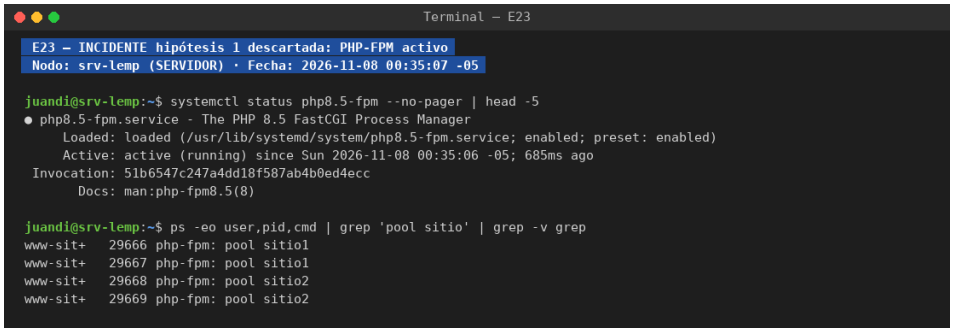


Figura 21. PHP-FPM activo y workers de los dos pools vivos (E23; salida real de capturar.sh, con hostname y fecha)

El servicio está activo y hay workers de *pool sitio2*: el proceso existe, el problema está entre Nginx y el pool.

11.4 Hipótesis 2: Nginx no logra conectarse al socket de sitio2 — confirmada

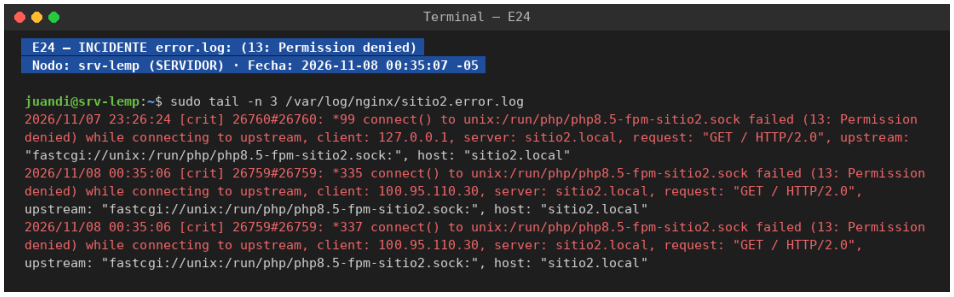


Figura 22. error.log de sitio2: connect() … failed (13: Permission denied) (E24; salida real de capturar.sh, con hostname y fecha)

El errno es la pista clave: 13 (Permission denied) significa que el socket existe pero no hay permiso; con el servicio detenido habría sido 2 (No such file or directory), como en el incidente anterior.

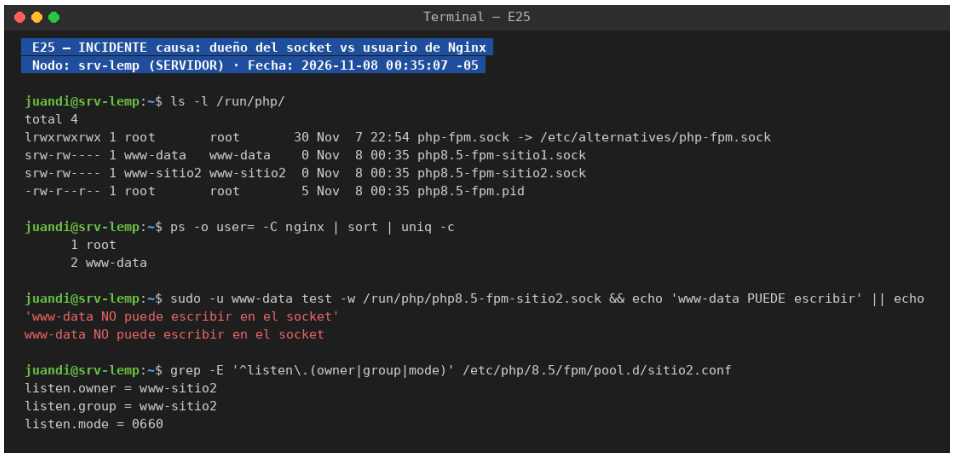


Figura 23. Causa: el socket de sitio2 pertenece a www-sitio2 y Nginx corre como www-data (E25; salida real de capturar.sh, con hostname y fecha)

El script de monitoreo también lo detectó: reportó *sitio=sitio2.local http=502* y *RESUMEN=ALERTA* (salida en *evidencias/salidas/F6_5b_monitor_detecta.txt*).

11.5 Causa raíz

listen.owner/listen.group del pool de *sitio2* asignaban el socket al usuario del pool. Con modo 0660, solo el dueño y su grupo pueden escribir; Nginx corre como *www-data*, que no es ninguno de los dos. Confundimos dos papeles distintos: *user/group* es quién ejecuta el PHP (eso aísla el sitio) y *listen.owner/group* es quién puede conectarse al socket (debe ser Nginx).

11.6 Solución y verificación

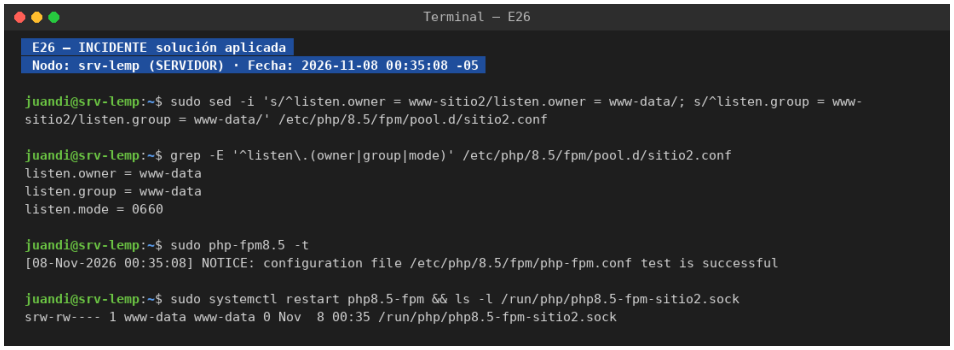


Figura 24. Solución: socket devuelto a www-data, validación y reinicio (E26; salida real de capturar.sh, con hostname y fecha)

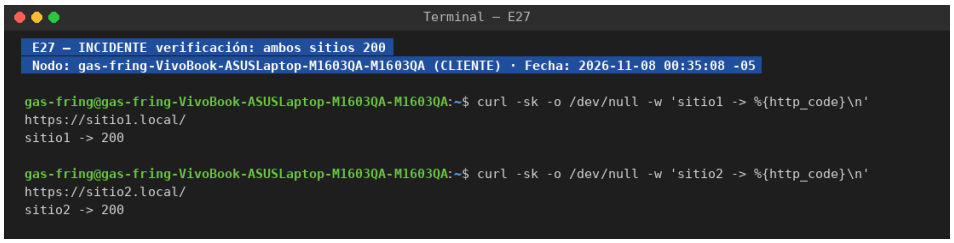


Figura 25. Verificación: ambos sitios 200, log con 200 y monitor en RESUMEN=OK (E27; salida real de capturar.sh, con hostname y fecha)

Alternativa válida: dejar el dueño y añadir *listen.acl_users = www-data*. Lo que no haríamos es *listen.mode = 0666*: funciona, pero cualquier usuario local podría ejecutar PHP como *www-sitio2*.

12. Pruebas de validación desde el segundo nodo

Todas las pruebas se hicieron desde el portátil (100.95.110.30) por Tailscale, mostrando el cuerpo de la respuesta y no solo los encabezados.

Prueba	**Resultado obtenido**
Resolución y ping a 100.79.166.31	Ambos nombres → 100.79.166.31; 0 % de pérdida
Contenido distinto por vhost	sitio1: “Panel de Administradores”; sitio2: “Portal de Lectores”
HTTP → HTTPS	301 Moved Permanently con *Location: https://sitioN.local/* en ambos
Encabezados	HSTS, X-Frame-Options, nosniff, Referrer-Policy, CSP; *Server: nginx* sin versión
TLS	1.0 y 1.1: *alert protocol version*; 1.2 y 1.3 aceptados
Archivos protegidos	*/.env* y */.git/config* → 403; */phpinfo.php* → 404
Acceso por IP o nombre desconocido	Conexión cerrada sin respuesta (444 / handshake rechazado)
Puertos (nmap)	22/80/443 open; 3306 closed; SSH por fuera de la VPN bloqueado
TLS (nmap ssl-enum-ciphers)	Solo TLSv1.2 y TLSv1.3; calificación mínima de cifrados: A

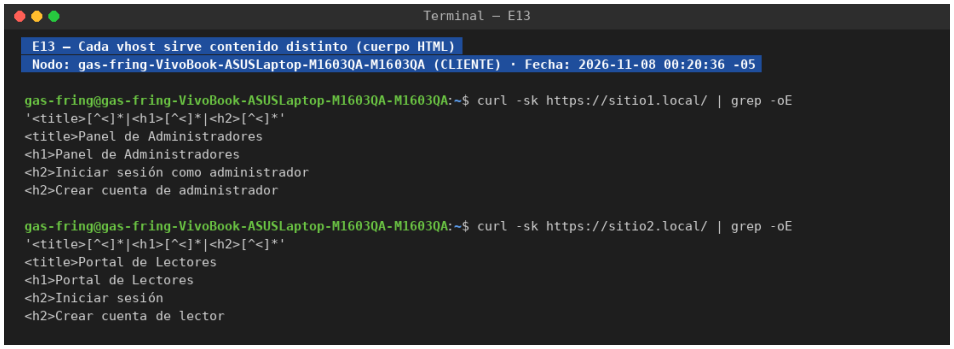


Figura 26. Cada vhost sirve contenido distinto (cuerpo HTML) (E13; salida real de capturar.sh, con hostname y fecha)

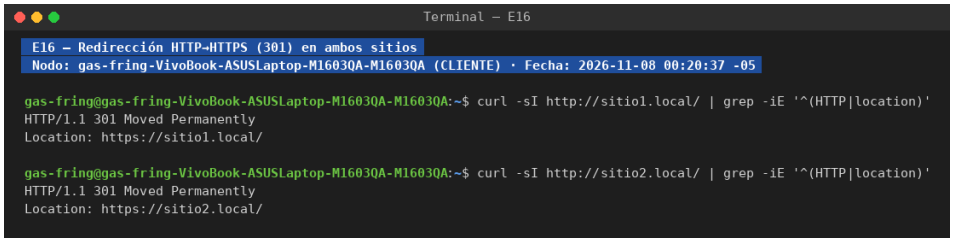


Figura 27. Redirección 301 de HTTP a HTTPS en ambos sitios (E16; salida real de capturar.sh, con hostname y fecha)

Sobre *curl -k*: omite la validación de confianza del certificado porque es autofirmado. La prueba confirma que la conexión va cifrada con TLS 1.2/1.3, pero no que una autoridad respalde la identidad del servidor.

13. Conclusiones

- El error más instructivo fue el del incidente: “aislar” un pool cambiando el dueño de su socket rompe el sitio. Aprendimos a separar quién ejecuta el código (*user*) de quién se conecta a él (*listen.owner*), y que en un socket Unix los permisos funcionan como en cualquier archivo.
- El número de error en el log ahorra tiempo: errno 2 (el socket no existe) apunta a un servicio caído y errno 13 (sin permiso) a una configuración de permisos. Leerlo antes de reiniciar servicios evitó perseguir la hipótesis equivocada.
- Validar antes de aplicar (*nginx -t*, *php-fpm8.5 -t*, *sshd -t*) nos salvó de varias caídas, pero no de los errores lógicos: la sintaxis del pool roto era válida. Hay que probar el comportamiento, no solo la sintaxis.
- Un firewall “por interfaz” es más fuerte que uno “por puerto”: con todo restringido a *tailscale0*, ni SSH ni la web existen fuera de la VPN, y eso hizo innecesario cambiar el puerto 22.
- Las pruebas también pueden engañar: la primera prueba de TLS 1.1 la rechazó el OpenSSL de nuestro propio cliente, no el servidor. Repetirla bajando el nivel de seguridad del cliente fue lo que demostró que el servidor lo rechaza.

- Un *umask* olvidado dejó los webroots en 700 y produjo un 403. Los permisos de Linux están detrás de casi todos los problemas que tuvimos.

14. Recomendaciones y trabajo futuro

- **Aplicación:** mover las credenciales de la BD fuera del webroot (archivo en */etc* con modo 640), hacer que el Portal de Lectores use *web_read*, retirar el formulario público de creación de administradores y proteger los formularios con token CSRF (eliminar con POST en vez de un enlace GET).
- **Certificados:** dominio real y certificado de una CA (Let's Encrypt), con renovación automática.
- **Fail2Ban:** bloquear IP tras intentos fallidos de SSH y patrones de abuso en los logs de Nginx.
- **Copias de seguridad:** *mariadb-dump* programado, guardado fuera del servidor y con pruebas periódicas de restauración.
- **Monitoreo con alertas:** Prometheus + node_exporter y alertas por correo o chat en vez de solo un log.
- **Automatización:** llevar toda la configuración a Ansible para reconstruir el servidor de forma idéntica.
- **WAF:** ModSecurity con el conjunto de reglas OWASP CRS delante de la aplicación.

15. Referencias

Canonical. (2026). Ubuntu Server documentation. https://documentation.ubuntu.com/server/

Canonical. (2026). Firewall — UFW. En Ubuntu Server documentation. https://documentation.ubuntu.com/server/how-to/security/firewalls/

MariaDB Foundation. (2026). GRANT. En MariaDB Knowledge Base. https://mariadb.com/kb/en/grant/

MariaDB Foundation. (2026). MariaDB Server documentation. https://mariadb.com/docs/

NGINX. (2026). nginx documentation. https://nginx.org/en/docs/

NGINX. (2026). Module ngx_http_fastcgi_module. https://nginx.org/en/docs/http/ngx_http_fastcgi_module.html

OpenBSD Project. (2026). sshd_config(5) — OpenSSH SSH daemon configuration file. https://man.openbsd.org/sshd_config

PHP Group. (2026). FastCGI Process Manager (FPM): Configuration. En PHP Manual. https://www.php.net/manual/en/install.fpm.configuration.php

Tailscale Inc. (2026). Tailscale documentation. https://tailscale.com/kb/

16. Anexos

Anexo A. Código PHP de la aplicación

A.1 sitio1.local — Panel de Administradores (/var/www/sitio1/index.php)

```
<?php
session_start();

/* ========================
   CONFIGURACIÓN MARIA DB
   ======================== */

$host = "127.0.0.1";
$db   = "proyecto_web";
$user = "web_admin";
$pass = "Admin123";

try {

    $pdo = new PDO(
        "mysql:host=$host;dbname=$db;charset=utf8mb4",
        $user,
        $pass
    );

    $pdo->setAttribute(
        PDO::ATTR_ERRMODE,
        PDO::ERRMODE_EXCEPTION
    );

} catch (PDOException $e) {

    die("Error de conexión con MariaDB.");

}


/* ========================
   CERRAR SESIÓN
   ======================== */

if (isset($_GET["logout"])) {

    session_unset();
    session_destroy();

    header("Location: index.php");
    exit;
}


$mensaje = "";
$error = "";
```

```
/* ========================
  LOGIN ADMIN
  ======================== */

if (
   $_SERVER["REQUEST_METHOD"] === "POST" &&
   isset($_POST["login"])
) {

   $correo = trim($_POST["correo"]);
   $password = $_POST["password"];


   $stmt = $pdo->prepare(
      "SELECT id, nombre, correo, password, rol
       FROM usuarios
       WHERE correo = ?"
   );

   $stmt->execute([$correo]);

   $usuario = $stmt->fetch(PDO::FETCH_ASSOC);


   if (
      $usuario &&
      password_verify(
         $password,
         $usuario["password"]
      ) &&
      $usuario["rol"] === "admin"
   ) {

      session_regenerate_id(true);

      $_SESSION["id"] = $usuario["id"];
      $_SESSION["nombre"] = $usuario["nombre"];
      $_SESSION["correo"] = $usuario["correo"];
      $_SESSION["rol"] = $usuario["rol"];

      header("Location: index.php");
      exit;

   } else {

      $error =
         "Credenciales incorrectas o el usuario no es administrador.";

   }
}


/* ========================
  CREAR ADMIN
  ========================
```

```
  El registro público crea
  administradores.

  Si quieres que solamente
  un administrador pueda crear
  otros administradores, puedes
  eliminar el formulario público
  de abajo.
*/

if (
    $_SERVER["REQUEST_METHOD"] === "POST" &&
    isset($_POST["crear_admin"])
) {

    $nombre = trim($_POST["nombre"]);
    $correo = trim($_POST["correo"]);
    $password = $_POST["password"];


    if (
        $nombre === "" ||
        $correo === "" ||
        $password === ""
    ) {

        $error = "Todos los campos son obligatorios.";

    } elseif (
        !filter_var(
            $correo,
            FILTER_VALIDATE_EMAIL
        )
    ) {

        $error = "El correo no es válido.";

    } elseif (strlen($password) < 6) {

        $error =
            "La contraseña debe tener al menos 6 caracteres.";

    } else {

        try {

            $hash = password_hash(
                $password,
                PASSWORD_DEFAULT
            );


            $stmt = $pdo->prepare(
                "INSERT INTO usuarios
                (nombre, correo, password, rol)
```

```
            VALUES (?, ?, ?, 'admin')"
        );

        $stmt->execute([
            $nombre,
            $correo,
            $hash
        ]);

        $mensaje =
            "Administrador creado correctamente.";

    } catch (PDOException $e) {

        $error =
            "No se pudo crear el administrador. El correo puede estar registrado.";

    }

  }
}


/* ========================
  TODAS LAS OPERACIONES
  SIGUIENTES REQUIEREN ADMIN
  ======================== */

$esAdmin =
  isset($_SESSION["id"]) &&
  isset($_SESSION["rol"]) &&
  $_SESSION["rol"] === "admin";


/* ========================
  CREAR USUARIO DESDE PANEL
  ======================== */

if (
  $esAdmin &&
  $_SERVER["REQUEST_METHOD"] === "POST" &&
  isset($_POST["crear_usuario"])
) {

  $nombre = trim($_POST["nombre"]);
  $correo = trim($_POST["correo"]);
  $password = $_POST["password"];
  $rol = $_POST["rol"];


  if (
      $nombre === "" ||
      $correo === "" ||
      $password === ""
  ) {
```

```
    $error = "Todos los campos son obligatorios.";

} elseif (
    !filter_var(
        $correo,
        FILTER_VALIDATE_EMAIL
    )
) {

    $error = "El correo no es válido.";

} elseif (
    !in_array(
        $rol,
        ["admin", "lector"],
        true
    )
) {

    $error = "Rol no válido.";

} elseif (strlen($password) < 6) {

    $error =
        "La contraseña debe tener al menos 6 caracteres.";

} else {

    try {

        $hash = password_hash(
            $password,
            PASSWORD_DEFAULT
        );


        $stmt = $pdo->prepare(
            "INSERT INTO usuarios
            (nombre, correo, password, rol)
            VALUES (?, ?, ?, ?)"
        );

        $stmt->execute([
            $nombre,
            $correo,
            $hash,
            $rol
        ]);

        $mensaje =
            "Usuario creado correctamente.";

    } catch (PDOException $e) {

        $error =
            "No se pudo crear el usuario. El correo puede estar registrado.";
```

```
        }
    }
}


/* ========================
   ACTUALIZAR USUARIO
   ======================== */

if (
    $esAdmin &&
    $_SERVER["REQUEST_METHOD"] === "POST" &&
    isset($_POST["actualizar"])
) {

    $id = (int)$_POST["id"];
    $nombre = trim($_POST["nombre"]);
    $correo = trim($_POST["correo"]);
    $rol = $_POST["rol"];


    if (
        $nombre === "" ||
        $correo === ""
    ) {

        $error =
            "Nombre y correo son obligatorios.";

    } elseif (
        !filter_var(
            $correo,
            FILTER_VALIDATE_EMAIL
        )
    ) {

        $error =
            "El correo no es válido.";

    } elseif (
        !in_array(
            $rol,
            ["admin", "lector"],
            true
        )
    ) {

        $error = "Rol no válido.";

    } else {

        /*
         * Evitamos que el administrador
         * se quite a sí mismo el rol.
         */
```

```
        if (
            $id === (int)$_SESSION["id"] &&
            $rol !== "admin"
        ) {

            $error =
                "No puedes cambiar tu propia cuenta a lector.";

        } else {

            try {

                $stmt = $pdo->prepare(
                    "UPDATE usuarios
                     SET nombre = ?,
                         correo = ?,
                         rol = ?
                     WHERE id = ?"
                );

                $stmt->execute([
                    $nombre,
                    $correo,
                    $rol,
                    $id
                ]);

                $mensaje =
                    "Usuario actualizado correctamente.";

            } catch (PDOException $e) {

                $error =
                    "No se pudo actualizar el usuario.";

            }
        }
    }
}


/* ========================
   ELIMINAR USUARIO
   ======================== */

if (
    $esAdmin &&
    isset($_GET["delete"])
) {

    $id = (int)$_GET["delete"];


    /*
     * No permitimos eliminar
```

```
     * la propia cuenta.
     */

    if ($id === (int)$_SESSION["id"]) {

        $error =
            "No puedes eliminar tu propia cuenta.";

    } else {

        try {

            $stmt = $pdo->prepare(
                "DELETE FROM usuarios
                 WHERE id = ?"
            );

            $stmt->execute([$id]);

            $mensaje =
                "Usuario eliminado correctamente.";

        } catch (PDOException $e) {

            $error =
                "No se pudo eliminar el usuario.";

        }
    }
}


/* ========================
   USUARIO PARA EDITAR
   ======================== */

$editar = null;


if (
    $esAdmin &&
    isset($_GET["edit"])
) {

    $id = (int)$_GET["edit"];


    $stmt = $pdo->prepare(
        "SELECT id, nombre, correo, rol
         FROM usuarios
         WHERE id = ?"
    );

    $stmt->execute([$id]);

    $editar =
```

```
    $stmt->fetch(PDO::FETCH_ASSOC);
}


/* ========================
  LISTAR USUARIOS
  ========================

  SOLO ADMIN
*/

$usuarios = [];


if ($esAdmin) {

   $stmt = $pdo->query(
      "SELECT id, nombre, correo, rol
       FROM usuarios
       ORDER BY id"
   );

   $usuarios =
      $stmt->fetchAll(PDO::FETCH_ASSOC);
}

?>

<!DOCTYPE html>

<html lang="es">

<head>

   <meta charset="UTF-8">

   <meta name="viewport"
        content="width=device-width, initial-scale=1.0">

   <title>Panel de Administradores</title>


   <style>

      body {
         font-family: Arial, sans-serif;
         background: #f4f6f8;
         margin: 0;
         padding: 40px;
      }

      .container {
         max-width: 1100px;
         margin: auto;
      }
```

```
.card {
    background: white;
    padding: 25px;
    margin-bottom: 25px;
    border-radius: 8px;
    box-shadow: 0 2px 8px rgba(0,0,0,0.1);
}

input,
select {
    display: block;
    width: 300px;
    max-width: 100%;
    padding: 10px;
    margin: 10px 0;
    box-sizing: border-box;
}

button {
    padding: 10px 18px;
    background: #2563eb;
    color: white;
    border: none;
    border-radius: 5px;
    cursor: pointer;
}

button:hover {
    background: #1d4ed8;
}

table {
    width: 100%;
    border-collapse: collapse;
}

th,
td {
    padding: 10px;
    border: 1px solid #ddd;
    text-align: left;
}

th {
    background: #e5e7eb;
}

.error {
    color: #b91c1c;
    background: #fee2e2;
    padding: 10px;
}

.success {
    color: #166534;
    background: #dcfce7;
```

```
            padding: 10px;
        }

        .logout {
            color: #dc2626;
        }

        .danger {
            color: #dc2626;
        }

    </style>

</head>


<body>

<div class="container">


    <div class="card">

        <h1>Panel de Administradores</h1>

        <p>
            Sitio 2 - Administración del sistema
        </p>

    </div>


    <?php if ($mensaje !== ""): ?>

        <div class="card success">
            <?= htmlspecialchars($mensaje) ?>
        </div>

    <?php endif; ?>


    <?php if ($error !== ""): ?>

        <div class="card error">
            <?= htmlspecialchars($error) ?>
        </div>

    <?php endif; ?>


    <?php if (!$esAdmin): ?>


        <!-- ========================
             LOGIN
             ======================== -->
```

```
<div class="card">

   <h2>Iniciar sesión como administrador</h2>

   <form method="POST">

      <input
         type="email"
         name="correo"
         placeholder="Correo"
         required
      >

      <input
         type="password"
         name="password"
         placeholder="Contraseña"
         required
      >

      <button
         type="submit"
         name="login"
      >
         Iniciar sesión
      </button>

   </form>

</div>


<!-- ==========================
    REGISTRAR ADMIN
    ========================== -->

<div class="card">

   <h2>Crear cuenta de administrador</h2>

   <form method="POST">

      <input
         type="text"
         name="nombre"
         placeholder="Nombre completo"
         required
      >

      <input
         type="email"
         name="correo"
         placeholder="Correo"
         required
      >
```

```
        <input
          type="password"
          name="password"
          placeholder="Contraseña"
          minlength="6"
          required
        >

        <button
          type="submit"
          name="crear_admin"
        >
          Crear administrador
        </button>

      </form>

    </div>


<?php else: ?>


    <!-- ========================
         ADMIN LOGUEADO
         ======================== -->

    <div class="card">

      <h2>
        Bienvenido,
        <?= htmlspecialchars($_SESSION["nombre"]) ?>
      </h2>

      <p>
        Correo:
        <?= htmlspecialchars($_SESSION["correo"]) ?>
      </p>

      <p>
        Rol:
        <strong>ADMINISTRADOR</strong>
      </p>

      <p>
        <a
          class="logout"
          href="?logout=1"
        >
          Cerrar sesión
        </a>
      </p>

    </div>
```

```
<!-- ========================
     CREAR USUARIO
     ======================== -->

<div class="card">

    <h2>Crear usuario</h2>

    <form method="POST">

        <input
            type="text"
            name="nombre"
            placeholder="Nombre completo"
            required
        >

        <input
            type="email"
            name="correo"
            placeholder="Correo"
            required
        >

        <input
            type="password"
            name="password"
            placeholder="Contraseña"
            minlength="6"
            required
        >

        <select name="rol" required>

            <option value="lector">
                Lector
            </option>

            <option value="admin">
                Administrador
            </option>

        </select>

        <button
            type="submit"
            name="crear_usuario"
        >
            Crear usuario
        </button>

    </form>

</div>
```

```
<!-- ========================
    EDITAR
    ======================== -->

<?php if ($editar): ?>

   <div class="card">

      <h2>Editar usuario</h2>

      <form method="POST">

         <input
            type="hidden"
            name="id"
            value="<?= htmlspecialchars($editar["id"]) ?>"
         >

         <label>
            Nombre
         </label>

         <input
            type="text"
            name="nombre"
            value="<?= htmlspecialchars($editar["nombre"]) ?>"
            required
         >

         <label>
            Correo
         </label>

         <input
            type="email"
            name="correo"
            value="<?= htmlspecialchars($editar["correo"]) ?>"
            required
         >

         <label>
            Rol
         </label>

         <select name="rol">

            <option
               value="lector"
               <?= $editar["rol"] === "lector"
                  ? "selected"
                  : "" ?>
            >
               Lector
            </option>
```

```
            <option
               value="admin"
               <?= $editar["rol"] === "admin"
                  ? "selected"
                  : "" ?>
            >
               Administrador
            </option>

         </select>

         <button
            type="submit"
            name="actualizar"
         >
            Guardar cambios
         </button>

      </form>

   </div>

<?php endif; ?>


<!-- ========================
    TABLA
    ======================== -->

<div class="card">

   <h2>Usuarios registrados</h2>

   <table>

      <tr>

         <th>ID</th>
         <th>Nombre</th>
         <th>Correo</th>
         <th>Rol</th>
         <th>Acciones</th>

      </tr>


      <?php foreach ($usuarios as $usuario): ?>

         <tr>

            <td>
               <?= htmlspecialchars($usuario["id"]) ?>
            </td>

            <td>
               <?= htmlspecialchars($usuario["nombre"]) ?>
```

```
                </td>

                <td>
                    <?= htmlspecialchars($usuario["correo"]) ?>
                </td>

                <td>
                    <?= htmlspecialchars($usuario["rol"]) ?>
                </td>

                <td>

                    <a
                        href="?edit=<?= $usuario["id"] ?>"
                    >
                        Editar
                    </a>


                    <?php if (
                        $usuario["id"]
                        != $_SESSION["id"]
                    ): ?>

                        |

                        <a
                            class="danger"
                            href="?delete=<?= $usuario["id"] ?>"
                            onclick="return confirm('¿Seguro que quieres eliminar este usuario?');"
                        >
                            Eliminar
                        </a>

                    <?php endif; ?>

                </td>

            </tr>

        <?php endforeach; ?>

    </table>

</div>


<?php endif; ?>


</div>

</body>

</html>
```

A.2 sitio2.local — Portal de Lectores (/var/www/sitio2/index.php)

```
<?php
session_start();

/* ========================
  CONFIGURACIÓN MARIA DB
  ======================== */

$host = "127.0.0.1";
$db   = "proyecto_web";
$user = "web_admin";
$pass = "Admin123";

try {
    $pdo = new PDO(
        "mysql:host=$host;dbname=$db;charset=utf8mb4",
        $user,
        $pass
    );

    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

} catch (PDOException $e) {
    die("Error de conexión con MariaDB.");
}


/* ========================
  CERRAR SESIÓN
  ======================== */

if (isset($_GET["logout"])) {

    session_unset();
    session_destroy();

    header("Location: index.php");
    exit;
}


/* ========================
  VARIABLES
  ======================== */

$mensaje = "";
$error = "";


/* ========================
  CREAR CUENTA DE LECTOR
  ======================== */

if ($_SERVER["REQUEST_METHOD"] === "POST"
    && isset($_POST["crear_lector"])) {
```

```
    $nombre = trim($_POST["nombre"]);
    $correo = trim($_POST["correo"]);
    $password = $_POST["password"];

    if ($nombre === "" || $correo === "" || $password === "") {

        $error = "Todos los campos son obligatorios.";

    } elseif (!filter_var($correo, FILTER_VALIDATE_EMAIL)) {

        $error = "El correo no es válido.";

    } elseif (strlen($password) < 6) {

        $error = "La contraseña debe tener al menos 6 caracteres.";

    } else {

        try {

            $hash = password_hash(
                $password,
                PASSWORD_DEFAULT
            );

            $stmt = $pdo->prepare(
                "INSERT INTO usuarios
                (nombre, correo, password, rol)
                VALUES (?, ?, ?, 'lector')"
            );

            $stmt->execute([
                $nombre,
                $correo,
                $hash
            ]);

            $mensaje = "Cuenta de lector creada correctamente.";

        } catch (PDOException $e) {

            $error = "No se pudo crear la cuenta. El correo puede estar registrado.";

        }
    }
}


/* ========================
   INICIAR SESIÓN
   ======================== */

if ($_SERVER["REQUEST_METHOD"] === "POST"
    && isset($_POST["login"])) {
```

```
    $correo = trim($_POST["correo"]);
    $password = $_POST["password"];

    $stmt = $pdo->prepare(
        "SELECT id, nombre, correo, password, rol
         FROM usuarios
         WHERE correo = ?"
    );

    $stmt->execute([$correo]);

    $usuario = $stmt->fetch(PDO::FETCH_ASSOC);


    if (
        $usuario &&
        password_verify($password, $usuario["password"]) &&
        $usuario["rol"] === "lector"
    ) {

        session_regenerate_id(true);

        $_SESSION["id"] = $usuario["id"];
        $_SESSION["nombre"] = $usuario["nombre"];
        $_SESSION["correo"] = $usuario["correo"];
        $_SESSION["rol"] = $usuario["rol"];

        header("Location: index.php");
        exit;

    } else {

        $error = "Credenciales incorrectas o el usuario no es lector.";

    }
}


/* ========================
   CREAR OTRO LECTOR
   ========================

   SOLO puede hacerlo un lector
   que ya haya iniciado sesión.
*/

if (
    isset($_SESSION["id"]) &&
    $_SESSION["rol"] === "lector" &&
    $_SERVER["REQUEST_METHOD"] === "POST" &&
    isset($_POST["crear_lector_sesion"])
) {

    $nombre = trim($_POST["nombre"]);
    $correo = trim($_POST["correo"]);
    $password = $_POST["password"];
```

```
    if ($nombre === "" || $correo === "" || $password === "") {

        $error = "Todos los campos son obligatorios.";

    } elseif (!filter_var($correo, FILTER_VALIDATE_EMAIL)) {

        $error = "El correo no es válido.";

    } elseif (strlen($password) < 6) {

        $error = "La contraseña debe tener al menos 6 caracteres.";

    } else {

        try {

            $hash = password_hash(
                $password,
                PASSWORD_DEFAULT
            );

            $stmt = $pdo->prepare(
                "INSERT INTO usuarios
                (nombre, correo, password, rol)
                VALUES (?, ?, ?, 'lector')"
            );

            $stmt->execute([
                $nombre,
                $correo,
                $hash
            ]);

            $mensaje = "Nuevo lector creado correctamente.";

        } catch (PDOException $e) {

            $error = "No se pudo crear el lector. El correo puede estar registrado.";

        }
    }
}


/* ========================
   OBTENER USUARIOS
   ========================

   SOLO si hay sesión de lector.
*/

$usuarios = [];

if (
    isset($_SESSION["id"]) &&
```

```
    $_SESSION["rol"] === "lector"
) {

    $stmt = $pdo->query(
        "SELECT id, nombre, correo, rol
         FROM usuarios
         ORDER BY id"
    );

    $usuarios = $stmt->fetchAll(PDO::FETCH_ASSOC);
}

?>

<!DOCTYPE html>

<html lang="es">

<head>

    <meta charset="UTF-8">

    <meta name="viewport"
          content="width=device-width, initial-scale=1.0">

    <title>Portal de Lectores</title>

    <style>

        body {
            font-family: Arial, sans-serif;
            background: #f4f6f8;
            margin: 0;
            padding: 40px;
        }

        .container {
            max-width: 1000px;
            margin: auto;
        }

        .card {
            background: white;
            padding: 25px;
            margin-bottom: 25px;
            border-radius: 8px;
            box-shadow: 0 2px 8px rgba(0,0,0,0.1);
        }

        input {
            display: block;
            width: 300px;
            max-width: 100%;
            padding: 10px;
            margin: 10px 0;
            box-sizing: border-box;
```

```
    }

    button {
      padding: 10px 18px;
      background: #2563eb;
      color: white;
      border: none;
      border-radius: 5px;
      cursor: pointer;
    }

    button:hover {
      background: #1d4ed8;
    }

    table {
      width: 100%;
      border-collapse: collapse;
    }

    th, td {
      padding: 10px;
      border: 1px solid #ddd;
      text-align: left;
    }

    th {
      background: #e5e7eb;
    }

    .error {
      color: #b91c1c;
      background: #fee2e2;
      padding: 10px;
    }

    .success {
      color: #166534;
      background: #dcfce7;
      padding: 10px;
    }

    .logout {
      color: #dc2626;
    }

  </style>

</head>

<body>

<div class="container">

  <div class="card">
```

```
    <h1>Portal de Lectores</h1>

    <p>
        Sitio 1 - Sistema de consulta de usuarios
    </p>

</div>


<?php if ($mensaje !== ""): ?>

    <div class="card success">
        <?= htmlspecialchars($mensaje) ?>
    </div>

<?php endif; ?>


<?php if ($error !== ""): ?>

    <div class="card error">
        <?= htmlspecialchars($error) ?>
    </div>

<?php endif; ?>


<?php if (!isset($_SESSION["id"])): ?>

    <!-- ========================
         LOGIN
         ======================== -->

    <div class="card">

        <h2>Iniciar sesión</h2>

        <form method="POST">

            <input
                type="email"
                name="correo"
                placeholder="Correo"
                required
            >

            <input
                type="password"
                name="password"
                placeholder="Contraseña"
                required
            >

            <button
                type="submit"
                name="login"
```

```
            >
                Iniciar sesión
            </button>

        </form>

    </div>


    <!-- ========================
        REGISTRO
        ======================== -->

    <div class="card">

        <h2>Crear cuenta de lector</h2>

        <form method="POST">

            <input
                type="text"
                name="nombre"
                placeholder="Nombre completo"
                required
            >

            <input
                type="email"
                name="correo"
                placeholder="Correo"
                required
            >

            <input
                type="password"
                name="password"
                placeholder="Contraseña"
                minlength="6"
                required
            >

            <button
                type="submit"
                name="crear_lector"
            >
                Crear cuenta
            </button>

        </form>

    </div>


<?php else: ?>
```

```
<!-- ========================
     USUARIO LOGUEADO
     ======================== -->

<div class="card">

   <h2>
      Bienvenido,
      <?= htmlspecialchars($_SESSION["nombre"]) ?>
   </h2>

   <p>
      Correo:
      <?= htmlspecialchars($_SESSION["correo"]) ?>
   </p>

   <p>
      Rol:
      <strong>LECTOR</strong>
   </p>

   <p>
      <a
         class="logout"
         href="?logout=1"
      >
         Cerrar sesión
      </a>
   </p>

</div>


<!-- ========================
     CREAR LECTOR
     ======================== -->

<div class="card">

   <h2>Crear nuevo lector</h2>

   <form method="POST">

      <input
         type="text"
         name="nombre"
         placeholder="Nombre completo"
         required
      >

      <input
         type="email"
         name="correo"
         placeholder="Correo"
         required
      >
```

```
        <input
            type="password"
            name="password"
            placeholder="Contraseña"
            minlength="6"
            required
        >

        <button
            type="submit"
            name="crear_lector_sesion"
        >
            Crear lector
        </button>

    </form>

</div>


<!-- ========================
     LISTA DE USUARIOS
     ======================== -->

<div class="card">

    <h2>Usuarios registrados</h2>

    <table>

        <tr>
            <th>ID</th>
            <th>Nombre</th>
            <th>Correo</th>
            <th>Rol</th>
        </tr>

        <?php foreach ($usuarios as $usuario): ?>

            <tr>

                <td>
                    <?= htmlspecialchars($usuario["id"]) ?>
                </td>

                <td>
                    <?= htmlspecialchars($usuario["nombre"]) ?>
                </td>

                <td>
                    <?= htmlspecialchars($usuario["correo"]) ?>
                </td>

                <td>
                    <?= htmlspecialchars($usuario["rol"]) ?>
```

```
                </td>
            </tr>
        <?php endforeach; ?>
    </table>
</div>
<?php endif; ?>
</div>
</body>
</html>
```

Anexo B. Archivos de configuración finales

/etc/nginx/sites-available/sitio1

```
# /etc/nginx/sites-available/sitio1
server {
    listen 80;
    server_name sitio1.local;
    access_log /var/log/nginx/sitio1.access.log lemp;
    # Forzar HTTPS
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    http2 on;
    server_name sitio1.local;

    root /var/www/sitio1;
    index index.php index.html;

    ssl_certificate     /etc/nginx/ssl/sitios.crt;
    ssl_certificate_key /etc/nginx/ssl/sitios.key;

    access_log /var/log/nginx/sitio1.access.log lemp;
    error_log  /var/log/nginx/sitio1.error.log warn;

    include snippets/seguridad-sitio.conf;

    location / {
        try_files $uri $uri/ =404;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        # Pool PHP-FPM exclusivo de este sitio (usuario de sistema www-sitio1)
```

```
        fastcgi_pass unix:/run/php/php8.5-fpm-sitio1.sock;
    }
}
```

/etc/nginx/sites-available/sitio2

```
# /etc/nginx/sites-available/sitio2
server {
    listen 80;
    server_name sitio2.local;
    access_log /var/log/nginx/sitio2.access.log lemp;
    # Forzar HTTPS
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    http2 on;
    server_name sitio2.local;

    root /var/www/sitio2;
    index index.php index.html;

    ssl_certificate     /etc/nginx/ssl/sitios.crt;
    ssl_certificate_key /etc/nginx/ssl/sitios.key;

    access_log /var/log/nginx/sitio2.access.log lemp;
    error_log  /var/log/nginx/sitio2.error.log warn;

    include snippets/seguridad-sitio.conf;

    location / {
        try_files $uri $uri/ =404;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        # Pool PHP-FPM exclusivo de este sitio (usuario de sistema www-sitio2)
        fastcgi_pass unix:/run/php/php8.5-fpm-sitio2.sock;
    }
}
```

/etc/nginx/sites-available/00-rechazo

```
# /etc/nginx/sites-available/00-rechazo  — servidor por defecto
# Cualquier petición que no sea para sitio1.local / sitio2.local (p. ej. por IP directa o un
# nombre inventado) no recibe respuesta: no se revela qué sitios existen.
server {
    listen 80 default_server;
    listen 443 ssl default_server;
    server_name _;
    ssl_reject_handshake on;   # en 443 ni siquiera se completa TLS (no se muestra el certificado)
    access_log /var/log/nginx/rechazo.access.log lemp;
    return 444;                # 444 = cerrar la conexión sin respuesta
```

```
}
```

/etc/nginx/snippets/seguridad-sitio.conf

```
# /etc/nginx/snippets/seguridad-sitio.conf   (contexto server — incluir en cada vhost 443)

# Solo TLS modernos (TLS 1.0/1.1 quedan rechazados)
ssl_protocols TLSv1.2 TLSv1.3;
ssl_session_cache shared:SSL:10m;
ssl_session_timeout 10m;

# Encabezados de seguridad ("always" = también en respuestas de error)
add_header Strict-Transport-Security "max-age=31536000" always;
add_header X-Frame-Options "DENY" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
# CSP: solo recursos del propio sitio. 'unsafe-inline' en style/script porque la app del equipo
# usa <style> y un onclick="confirm(...)" en línea (sin él, "Eliminar" borraría sin confirmar).
add_header Content-Security-Policy "default-src 'self'; style-src 'self' 'unsafe-inline'; script-src 'self' 'unsafe-inline';
frame-ancestors 'none'; form-action 'self'" always;

# Tamaño máximo de cuerpo de petición
client_max_body_size 2m;

# Límite de tasa (zona definida en conf.d/00-hardening.conf)
limit_req zone=por_ip burst=20 nodelay;

# Bloquear archivos/directorios ocultos (.git, .env, .htaccess...)
location ~ /\. {
    deny all;
}
```

/etc/nginx/conf.d/00-hardening.conf

```
# /etc/nginx/conf.d/00-hardening.conf   (contexto http — se incluye desde nginx.conf)
# OJO: ssl_protocols y ssl_prefer_server_ciphers YA existen en el bloque http de
# /etc/nginx/nginx.conf en Ubuntu; repetirlos aquí da "directive is duplicate".
# Por eso el TLS se fija a nivel server en snippets/seguridad-sitio.conf.

# server_tokens: Ubuntu 26.04 ya trae "server_tokens build;" en nginx.conf (repetirlo aquí
# da "directive is duplicate"), así que se cambia allí a "off" (ver 04_hardening_servicio.sh).

# Límite de tasa por IP (lo usa cada vhost con limit_req)
limit_req_zone $binary_remote_addr zone=por_ip:10m rate=10r/s;
limit_req_status 429;

# Formato de log con tiempos: request_time (total) y upstream_response_time (lo que tardó PHP-FPM)
log_format lemp '$remote_addr - $remote_user [$time_local] "$host" "$request" '
                '$status $body_bytes_sent "$http_referer" "$http_user_agent" '
                'rt=$request_time urt=$upstream_response_time';
```

/etc/php/8.5/fpm/pool.d/sitio1.conf

```
; /etc/php/8.5/fpm/pool.d/sitio1.conf
[sitio1]
; Los procesos PHP de sitio1 corren con su propio usuario de sistema
user = www-sitio1
group = www-sitio1

; Socket exclusivo. Nginx (workers como www-data) debe poder escribir en él:
listen = /run/php/php8.5-fpm-sitio1.sock
listen.owner = www-data
listen.group = www-data
listen.mode = 0660

pm = dynamic
pm.max_children = 5
pm.start_servers = 2
pm.min_spare_servers = 1
pm.max_spare_servers = 3

; Solo puede leer su propio webroot y el directorio de sesiones (no el del otro sitio)
php_admin_value[open_basedir] = /var/www/sitio1:/var/lib/php/sessions:/tmp
catch_workers_output = yes
```

/etc/php/8.5/fpm/pool.d/sitio2.conf

```
; /etc/php/8.5/fpm/pool.d/sitio2.conf
[sitio2]
; Los procesos PHP de sitio2 corren con su propio usuario de sistema
user = www-sitio2
group = www-sitio2

; Socket exclusivo. Nginx (workers como www-data) debe poder escribir en él:
listen = /run/php/php8.5-fpm-sitio2.sock
listen.owner = www-data
listen.group = www-data
listen.mode = 0660

pm = dynamic
pm.max_children = 5
pm.start_servers = 2
pm.min_spare_servers = 1
pm.max_spare_servers = 3

; Solo puede leer su propio webroot y el directorio de sesiones (no el del otro sitio)
php_admin_value[open_basedir] = /var/www/sitio2:/var/lib/php/sessions:/tmp
catch_workers_output = yes
```

/etc/php/8.5/fpm/conf.d/99-hardening.ini

```
; /etc/php/8.5/fpm/conf.d/99-hardening.ini
; No anunciar la versión de PHP (encabezado X-Powered-By)
expose_php = Off
; Errores al log, nunca al navegador
```

```
display_errors = Off
display_startup_errors = Off
log_errors = On
; Cookies de sesión: solo por HTTPS, inaccesibles a JavaScript, sin IDs impuestos
session.cookie_secure = 1
session.cookie_httponly = 1
session.cookie_samesite = Strict
session.use_strict_mode = 1
```

/etc/ssh/sshd_config.d/01-hardening.conf

```
# /etc/ssh/sshd_config.d/01-hardening.conf
# El prefijo 01- es intencional: sshd usa el PRIMER valor que lee de cada directiva,
# y los archivos de sshd_config.d se leen en orden alfabético (50-cloud-init.conf
# puede traer "PasswordAuthentication yes").
# ⚠️ Aplicar SOLO después de confirmar que el login por llave funciona.

PermitRootLogin no
PubkeyAuthentication yes
PasswordAuthentication no
KbdInteractiveAuthentication no
PermitEmptyPasswords no
MaxAuthTries 3
LoginGraceTime 30
X11Forwarding no
# Ajustar con los usuarios reales que administran el servidor:
AllowUsers juandi
```

/etc/cron.d/lemp-monitor

```
*/5 * * * * root /usr/local/bin/monitor_lemp.sh >> /var/log/lemp-monitor.log 2>&1
```

/etc/logrotate.d/lemp-monitor

```
/var/log/lemp-monitor.log {
    su root syslog
    weekly
    rotate 4
    compress
    missingok
    notifempty
}
```

/etc/security/pwquality.conf (líneas activas)

```
minlen = 12
minclass = 3
```

Reglas de UFW (ufw status verbose)

```
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp on tailscale0       ALLOW IN    Anywhere                   # SSH solo por Tailscale
80/tcp on tailscale0       ALLOW IN    Anywhere                   # HTTP (301 a HTTPS) solo por Tailscale
443/tcp on tailscale0      ALLOW IN    Anywhere                   # HTTPS solo por Tailscale
41641/udp                  ALLOW IN    Anywhere                   # Tailscale WireGuard directo (peer-to-peer)
22/tcp (v6) on tailscale0  ALLOW IN    Anywhere (v6)              # SSH solo por Tailscale
80/tcp (v6) on tailscale0  ALLOW IN    Anywhere (v6)              # HTTP (301 a HTTPS) solo por Tailscale
443/tcp (v6) on tailscale0 ALLOW IN    Anywhere (v6)              # HTTPS solo por Tailscale
41641/udp (v6)             ALLOW IN    Anywhere (v6)              # Tailscale WireGuard directo (peer-to-peer)
```

Anexo C. Scripts SQL y datos reales

C.1 Creación de la base, la tabla, los datos iniciales y los usuarios

```
CREATE DATABASE IF NOT EXISTS proyecto_web CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
USE proyecto_web;
CREATE TABLE IF NOT EXISTS usuarios (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nombre VARCHAR(100) NOT NULL,
  correo VARCHAR(120) NOT NULL UNIQUE,
  password VARCHAR(255) NOT NULL,
  rol ENUM('admin','lector') NOT NULL DEFAULT 'lector',
  creado TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
INSERT IGNORE INTO usuarios (nombre, correo, password, rol) VALUES
  ('Juan Diego López',      'jdlopez@proyecto.local',  '<hash bcrypt de password_hash()>', 'admin'),
  ('Luis Camilo Vivas Cruz','lcvivas@proyecto.local',  '<hash bcrypt de password_hash()>', 'admin'),
  ('Dylan Mancipe',         'dmancipe@proyecto.local', '<hash bcrypt de password_hash()>', 'lector');
CREATE USER IF NOT EXISTS 'web_admin'@'localhost' IDENTIFIED BY '<contraseña>';
CREATE USER IF NOT EXISTS 'web_read'@'localhost'  IDENTIFIED BY '<contraseña>';
GRANT SELECT, INSERT, UPDATE, DELETE ON proyecto_web.* TO 'web_admin'@'localhost';
GRANT SELECT ON proyecto_web.* TO 'web_read'@'localhost';
FLUSH PRIVILEGES;
```

C.2 Datos reales de la tabla usuarios (consultados con web_read)

```
$ mariadb -u web_read -p proyecto_web -e "SELECT CURRENT_USER(); SELECT id,nombre,correo,rol FROM
usuarios;"
CURRENT_USER()
web_read@localhost
id    nombre    correo    rol
1    Juan Diego López    jdlopez@proyecto.local    admin
2    Luis Camilo Vivas Cruz    lcvivas@proyecto.local    admin
3    Dylan Mancipe    dmancipe@proyecto.local    lector
```

Nota: después del despliegue la contraseña de *web_admin* se cambió a la que trae fija el código del equipo (ver bitácora). Las cuentas creadas desde la aplicación se agregan a esta tabla.

Anexo D. Bitácora técnica

Fecha-hora	Nodo	Comando / acción	Resultado	Error y solución
2026-11-07 21:40	portátil (gas-fring)	*VBoxManage modifyvm rufus --memory 3072 + startvm --type headless*	VM *rufus* (hostname *nodoA*) arranca; SSH por NAT 127.0.0.1:2222	RAM bajada de 8 GB a 3 GB porque el anfitrión solo tenía ~5 GB libres
2026-11-07 21:55	portátil → nodoA	*ssh-copy-id -i ~/.ssh/reto2_ed25519 -p 2222 juancho@localhost*	Acceso por llave ed25519	—
2026-11-07 21:58	nodoA	*/etc/sudoers.d/90-reto2-temporal* (NOPASSWD para juancho)	sudo no interactivo para la automatización	**Temporal**: se elimina al cerrar el proyecto
2026-11-07 22:04	nodoA	*sudo bash scripts/00_reconocimiento.sh*	Salida en *evidencias/salidas/F0_estado_servidor.txt*	Nginx 1.28.3 instalado pero inactivo; PHP y MariaDB no instalados; la VM aloja otros proyectos (ver Fase 0)
2026-11-07 22:05	nodoA	Decisión del equipo	Se **descarta** *nodoA* como servidor: aloja otros proyectos (cluster pcs/corosync, NFS, PostgreSQL, MongoDB, Pollos Hermanos) y tiene Tailscale Funnel ocupando 443	Un firewall solo 22/80/443 los rompería → VM dedicada
2026-11-07 22:11	portátil	*crear_vms.sh* (VBoxManage unattended, Ubuntu Server 26.04.1, ISO verificada SHA-256)	VM *srv-lemp* 2 GB / 2 vCPU / 25 GB, NAT con SSH en 127.0.0.1:2225	El cliente será el propio portátil (segundo nodo por Tailscale); la VM *cli-lemp* creada por error se borró
2026-11-07 22:35	srv-lemp	Instalación desatendida	Sistema instalado; falló el comando post-instalación (*curtin in-target* no existe dentro del chroot de VirtualBox)	Se entró por consola y se instaló *openssh-server*, la llave ed25519 y sudo temporal
2026-11-07 22:38	portátil	*ssh-keygen -R '[127.0.0.1]:2225'*	—	Aviso "REMOTE HOST IDENTIFICATION HAS CHANGED": la llave vieja era del entorno del instalador; se comprobó que la nueva huella (SHA256:gT2X55ON…) coincide con la que mostró la consola
2026-11-07 22:40	srv-lemp	Repositorio oficial de Tailscale + *apt install tailscale + tailscale up --hostname=srv-lemp*	IP Tailscale **100.79.166.31**; ping desde el portátil (100.95.110.30) OK	—
2026-11-07 22:42	srv-lemp	*sudo bash scripts/00_reconocimiento.sh* (Fase 0)	Salida en *evidencias/salidas/F0_estado_servidor.txt*: sistema limpio, sin LEMP	Zona horaria en UTC (hay que pasarla a America/Bogota)
2026-11-07 22:50	srv-lemp	*timedatectl set-timezone America/Bogota + apt full-upgrade*	Hora local Colombia; sistema al día	—
2026-11-07 22:53	srv-lemp	*apt install nginx mariadb-server php8.5-fpm php8.5-mysql php8.5-cli*	Nginx 1.28.3, PHP 8.5.4, MariaDB 11.8.6 activos y habilitados	—
2026-11-07 22:54	srv-lemp	SQL: BD *proyecto_web*, tabla *usuarios*, 3 registros (contraseñas con *password_hash*), *web_admin* (SELECT/INSERT/UPDATE/DELETE) y *web_read* (SELECT)	*SHOW GRANTS* confirma privilegios diferenciados; contraseñas de BD aleatorias en */root/reto2_credenciales_bd.txt* (600)	—
2026-11-07 22:54	srv-lemp	*openssl req -x509 … -addext subjectAltName=DNS:sitio1.local,DNS:sitio2.local*	Certificado autofirmado válido hasta 2027-11-08 con SAN para ambos sitios	—

2026-11-07 22:54	srv-lemp	vhosts *sitio1* / *sitio2* (80 → 301, 443 + PHP-FPM), *nginx -t*, *systemctl reload nginx*	Sintaxis OK; 301 a HTTPS en ambos	La primera prueba dio 200 porque aún respondían los workers viejos durante el reload
2026-11-07 22:58	srv-lemp	*curl -k https://sitioN.local/*	**403 Forbidden**	*umask 077* (puesto para el archivo de credenciales) se quedó activo y creó */var/www/sitioN* con modo 700 → Nginx (www-data) no podía entrar. Solución: *chmod 755* y limitar el umask a un subshell en el script. Tras el arreglo cada sitio devuelve su propio contenido
2026-11-07 23:09	portátil (cliente)	*echo "100.79.166.31 sitio1.local sitio2.local" | sudo tee -a /etc/hosts* + *curl* HTTP/HTTPS	Desde el segundo nodo: 301 a HTTPS y contenido distinto en cada sitio (*evidencias/salidas/F05_prueba_desde_cliente.txt*)	La primera prueba dio *000* porque se ejecutó antes de que quedara la entrada en */etc/hosts*; al repetir, OK
2026-11-07 23:10	portátil → srv-lemp	Acceso administrativo movido de NAT (127.0.0.1:2225) a SSH por Tailscale (100.79.166.31:22)	*SSH_CONNECTION* = 100.95.110.30 → 100.79.166.31:22	Necesario antes de cerrar todo lo que no entre por *tailscale0*
2026-11-07 23:10	srv-lemp	*ufw default deny incoming* / *allow outgoing*; *ufw allow in on tailscale0 to any port 22,80,443 proto tcp*; *ufw allow 41641/udp*; *ufw logging low*; *ufw --force enable*	UFW activo; solo 22/80/443 por *tailscale0* + 41641/udp (WireGuard directo de Tailscale)	Orden seguro: reglas primero, luego enable; sesión SSH nueva por Tailscale confirmada tras activar
2026-11-07 23:11	portátil (cliente)	Prueba de puertos con */dev/tcp* hacia 100.79.166.31 y SSH por NAT	22/80/443 abiertos por Tailscale; 3306 cerrado; SSH por NAT (enp0s3) → **timed out** (bloqueado); ambos sitios responden. Evidencia: *evidencias/salidas/F1_firewall.txt*	*nmap* no está instalado en el portátil (se instalará para la Fase 7)
2026-11-07 23:30	srv-lemp	Despliegue de la app del equipo **sin modificaciones**: *index(1).php* → */var/www/sitio1/index.php* (Panel de Administradores), *index.php* → */var/www/sitio2/index.php* (Portal de Lectores); dueño root:root, 644	SHA-256 idénticos a los originales; *php -l* sin errores; ambos sitios HTTP 200 desde el cliente y conectan a MariaDB	Páginas provisionales respaldadas como *index.php.provisional.<fecha>*
2026-11-07 23:30	srv-lemp	*ALTER USER 'web_admin'@'localhost' IDENTIFIED BY 'Admin123'*	La app conecta (el código trae esa contraseña fija)	Decisión del equipo: usar el código tal cual
2026-11-07 23:20	srv-lemp	Copia *sshd_config.bak.<fecha>*; *config/ssh/01-hardening.conf* → */etc/ssh/sshd_config.d/* (AllowUsers juandi); *sshd -t*; *systemctl daemon-reload*; *restart ssh*	*sshd -T*: permitrootlogin no, passwordauthentication no, maxauthtries 3, allowusers juandi	Login por llave probado ANTES y en sesión nueva DESPUÉS; consola VirtualBox como respaldo
2026-11-07 23:22	portátil (cliente)	Intento por contraseña y como root	Ambos: *Permission denied (publickey)*; por llave entra (*F2_ssh_cliente.txt*)	—
2026-11-07 23:24	srv-lemp	*apt install libpam-pwquality*; *pwquality.conf*: minlen = 12, minclass = 3	*pam_pwquality.so* activo en *common-password*; *pwscore* rechaza clave corta	*pwscore* no venía instalado → *apt install libpwquality-tools*
2026-11-07 23:25	srv-lemp	*apt full-upgrade*; *unattended-upgrades* + *dpkg-reconfigure*	0 paquetes pendientes; actualizaciones automáticas diarias activas	Avisos de debconf por no tener terminal (no afectan)
2026-11-07 23:23	srv-lemp	*useradd --system --shell /usr/sbin/nologin www-sitio1/www-sitio2*; pools *sitio1.conf* / *sitio2.conf* en */etc/php/8.5/fpm/pool.d/* (socket propio, *open_basedir* = su webroot + sesiones + /tmp); *php-fpm8.5 -t*; vhosts → *php8.5-fpm-sitioN.sock*;	Workers *pool sitio1* como www-sitio1 y *pool sitio2* como www-sitio2; sockets www-data 0660; ambos sitios 200	Se ajustó *open_basedir* del borrador: se quitó */etc/proyecto_web/*.ini* porque el código del equipo no lo usa

		nginx -t; reload		
2026-11-07 23:23	srv-lemp	Pool por defecto *www.conf* movido a */root/*	Solo quedan los dos pools de los sitios	Nadie lo usaba; menos procesos y superficie
2026-11-07 23:23	portátil → srv-lemp	Script temporal en sitio2 que intenta leer */var/www/sitio1/index.php* (borrado al terminar)	*BLOQUEADO … Operation not permitted* (open_basedir); evidencia *F3_aislamiento_pools.txt*	—
2026-11-07 23:25	srv-lemp	Respaldos *index.php.provisional.** movidos de los webroots a */root/respaldos/*	Webroots solo con *index.php*	Dentro del webroot Nginx los habría servido como texto plano
2026-11-07 23:24	srv-lemp	Respaldo */etc/nginx → /root/respaldos/*; *server_tokens build → off* en *nginx.conf*; *conf.d/00-hardening.conf* (log_format *lemp*, *limit_req_zone*); *snippets/seguridad-sitio.conf* (TLS 1.2/1.3, HSTS, X-Frame-Options, nosniff, Referrer-Policy, CSP, bloqueo de ocultos, límite de tasa, *client_max_body_size*); vhosts finales con *http2* y logs por sitio; servidor por defecto *00-rechazo* (444 + *ssl_reject_handshake*)	*nginx -t* OK; reload	Ubuntu 26.04 ya trae *server_tokens build;* en *nginx.conf*: ponerlo en conf.d habría dado "directive is duplicate" → se cambió en su sitio. CSP con *'unsafe-inline'* en script/style porque la app del equipo usa *<style>* y *onclick=confirm()* en línea
2026-11-07 23:24	srv-lemp	*conf.d/99-hardening.ini* (expose_php Off, display_errors Off, cookies secure/HttpOnly/SameSite=Strict, use_strict_mode); *php-fpm8.5 -t*; restart	Valores confirmados con *php-fpm8.5 -i*	—
2026-11-07 23:25	srv-lemp	MariaDB: *bind-address = 127.0.0.1*, *SELECT user,host FROM mysql.user*	Solo escucha en localhost; sin usuarios anónimos ni root remoto → no hizo falta *mariadb-secure-installation*	—
2026-11-07 23:25	portátil (cliente)	*curl -I*, *openssl s_client*, rutas ocultas, acceso por IP	Encabezados de seguridad presentes y *Server: nginx* sin versión; TLS 1.0/1.1 → *alert protocol version*, 1.2/1.3 OK; *.env*/*.git* → 403; *phpinfo.php* → 404; por IP → sin respuesta (444 / handshake rechazado). Evidencia *F4_hardening_cliente.txt*	La primera prueba de TLS 1.1 la rechazaba el OpenSSL del propio cliente ("no protocols available"); se repitió con *@SECLEVEL=0* para que respondiera el servidor
2026-11-07 23:25	srv-lemp	*monitor_lemp.sh → /usr/local/bin*; */etc/cron.d/lemp-monitor* (cada 5 min → */var/log/lemp-monitor.log*); */etc/logrotate.d/lemp-monitor*	Primera ejecución: 3 servicios active, ambos sitios 200, sockets presentes, *RESUMEN=OK*	logrotate rechazó la regla ("parent directory has insecure permissions") → se añadió *su root syslog*
2026-11-07 23:25	srv-lemp	Revisión de los 2 errores que reportó el monitor	Eran las pruebas de la Fase 4 (*/.env* y */.git/config* → "access forbidden by rule")	—
2026-11-07 23:26	srv-lemp	Logs por vhost con formato *lemp* (*rt=* / *urt=*), *journalctl -u php8.5-fpm*	Accesos del cliente 100.95.110.30 visibles por sitio con tiempos; evidencia *F5_monitoreo.txt*	Los logs nuevos los creó Nginx como root 644 → *chown www-data:adm* + *chmod 640* (igual que los de Ubuntu)
2026-11-07 23:26	portátil + srv-lemp	**Incidente (Fase 6)**: *listen.owner/group = www-sitio2* en el pool de sitio2 (respaldo en */root/respaldos/*); *php-fpm8.5 -t* (válido); restart	Cliente: sitio1 200, **sitio2 502 Bad Gateway**	Falla provocada a propósito
2026-11-07 23:26	srv-lemp	Hipótesis 1 "PHP-FPM caído": *systemctl status*, *ps … pool sitio*	**Descartada**: servicio active y workers de ambos pools vivos	—
2026-11-07 23:26	srv-lemp	*tail sitio2.error.log*; *ls -l /run/php/*; *ps -C nginx*; *sudo -u www-data test -w <socket>*	*connect() … php8.5-fpm-sitio2.sock failed (13: Permission denied)*; socket de sitio2 es www-sitio2:www-sitio2 0660 y Nginx corre como www-data → no puede escribir.	**Causa raíz**: el dueño del socket debe ser quien se conecta (Nginx), no el usuario del pool

			monitor_lemp.sh → RESUMEN=ALERTA	
2026-11-07 23:26	srv-lemp	*listen.owner/group = www-data*; *php-fpm8.5 -t*; restart	Socket www-data 0660; config idéntica al respaldo; cliente 200/200; monitor *RESUMEN=OK*. Evidencias *F6_*.txt*	—
2026-11-07 23:27	srv-lemp	*SHOW GRANTS*; como *web_read*: SELECT, INSERT, DELETE, DROP	SELECT OK; INSERT/DELETE/DROP → **ERROR 1142** (comando denegado). Evidencia *F8_privilegios_bd_E10_E11.txt*	—
2026-11-07 23:27	portátil (cliente)	*scripts/07_validacion_cliente.sh* (IP por defecto → 100.79.166.31; búsqueda de contenido por *<title>/<h1>*)	Resolución, ping, contenido distinto, 301, encabezados y archivos protegidos OK (*F7_validacion_cliente.txt*)	*nmap* no instalado en el cliente: pendiente *sudo apt install nmap* para E18/E19
2026-11-07 23:28	portátil	*scripts/capturar.sh* adaptado al sistema real (IP, usuario *juandi*, E07 con prueba de open_basedir, E11 sin pedir contraseña, E13 por título, E14/E15 según la app del equipo, E26 aplica la corrección, nuevo paso *provocar*)	Probado E07 y E11 en el servidor	E07: error de comillas anidadas en *php -r* → corregido
2026-11-07 23:30	portátil	*entregables/diagrama.py* (matplotlib)	*img/arquitectura.png* y *img/arquitectura_ficha.png*	Times New Roman no está instalada en el portátil → se usa Liberation Serif (métricamente equivalente) solo en las imágenes
2026-11-07 23:33	portátil	*entregables/generar_informe.py* (python-docx) + *libreoffice --convert-to pdf*	Informe con 16 secciones; cuerpo de 18 páginas (sin portada ni índice); solo Times New Roman en *document.xml* y *styles.xml*	Los estilos de tabla de la plantilla traían fuentes del tema (majorHAnsi) en formatos condicionales → se forzó TNR en todo *rFonts*. LibreOffice no rellena el campo TOC → índice estático en dos pasadas con páginas reales
2026-11-07 23:35	portátil	*entregables/generar_ficha.py*	Ficha técnica de 2 páginas, solo Times New Roman	—
2026-11-07 23:50	portátil (cliente)	*sudo apt install nmap* (lo ejecutó el usuario); *07_validacion_cliente.sh* de nuevo	*nmap*: 22/80/443 open, **3306 closed**; *ssl-enum-ciphers*: solo TLSv1.2/1.3, calificación mínima A	—
2026-11-08 00:35	portátil + srv-lemp	*capturar.sh* ejecutado para las 25 evidencias de terminal (servidor por SSH; incidente completo: *provocar* → E22 a E27)	Salidas reales con hostname y fecha en *evidencias/salidas/E*.txt*; incidente reproducido (sitio2 502 → errno 13 → corrección → 200/200, monitor *RESUMEN=OK*, pool idéntico al original)	E20 generaba tráfico desde el propio servidor (127.0.0.1) → se quitó y el tráfico se generó desde el cliente
2026-11-08 00:37	portátil	*scripts/render_terminal.py*: cada salida real → imagen con aspecto de terminal (*evidencias/capturas/E*.png*)	24 figuras en el informe; el pie aclara “salida real de capturar.sh, con hostname y fecha”	GNOME en Wayland no permite pantallazos automáticos; las imágenes reproducen el texto sin alterarlo
2026-11-08 00:53	portátil (cliente, navegador)	Capturas E14 (Panel de Administradores con sesión admin) y E15 (Portal de Lectores con sesión lector)	Cuentas creadas desde la app: id 4 (admin) e id 6 (lector); ambos sitios muestran los usuarios leídos de MariaDB	—

Anexo E. Script de monitoreo (/usr/local/bin/monitor_lemp.sh)

```
#!/usr/bin/env bash
# monitor_lemp.sh — Estado básico del stack LEMP. Instalar en /usr/local/bin/monitor_lemp.sh
# Uso manual:  sudo monitor_lemp.sh
# Programado:  cada 5 min con cron  ->  */5 * * * * root /usr/local/bin/monitor_lemp.sh >> /var/log/lemp-
monitor.log 2>&1
TS=$(date '+%F %T')
estado=0
for s in nginx php8.5-fpm mariadb; do
  st=$(systemctl is-active "$s")
  [ "$st" = active ] || estado=1
  printf '%s servicio=%s estado=%s\n' "$TS" "$s" "$st"
done
for n in 1 2; do
  sock=/run/php/php8.5-fpm-sitio$n.sock
  [ -S "$sock" ] && s_ok=presente || { s_ok=AUSENTE; estado=1; }
  code=$(curl -sk -o /dev/null -w '%{http_code}' --max-time 5 --resolve "sitio$n.local:443:127.0.0.1"
"https://sitio$n.local/")
  [ "$code" = 200 ] || estado=1
  printf '%s sitio=sitio%s.local http=%s socket=%s\n' "$TS" "$n" "$code" "$s_ok"
done
errs=$(tail -n 200 /var/log/nginx/sitio*.error.log 2>/dev/null | grep -c -E '\[(error|crit|alert|emerg)\]')
printf '%s errores_nginx_ultimas_200_lineas=%s\n' "$TS" "$errs"
[ "$estado" = 0 ] && echo "$TS RESUMEN=OK" || echo "$TS RESUMEN=ALERTA"
exit $estado
```